Programas PHP 4 Programación Orientada a Objetos

Programa To-Do-List

Archivos:

```
proyecto_poo/
├── Query.sql   # Querys de la página web
├── Database.php   # Clase para la conexión PDO a MySQL
├── Task.php       # Clase que representa el Modelo / Lógica de Tareas
└── index.php      # Vista principal e interacción con el usuario
```

Query.sql

```
CREATE DATABASE IF NOT EXISTS todo_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE todo_db;

CREATE TABLE IF NOT EXISTS tasks (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    completed TINYINT(1) DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Database.php

```
<?php
class Database {
    // Propiedades privadas: Nadie fuera de esta clase puede modificarlas directamente
    private $host = "localhost";
    private $db_name = "todo_db";
    private $username = "root";
    private $password = "";
    private $conn;

    // Método para obtener la conexión a la base de datos
    public function getConnection() {
        $this->conn = null;

        try {
            // Usamos PDO por seguridad y flexibilidad
            $this->conn = new PDO(
                "mysql:host=" . $this->host . ";dbname=" . $this->db_name,
                $this->username,
                $this->password
            );
```

```
            // Configurar PDO para reportar errores mediante Excepciones
            $this->conn->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);
        } catch(PDOException $exception) {
            echo "Error de conexión: " . $exception->getMessage();
        }

        return $this->conn;
    }
}
```

Task.php

```
<?php
class Task {
    // Conexión y nombre de la tabla
    private $conn;
    private $table_name = "tasks";

    // Propiedades del Objeto
    public $id;
    public $title;
    public $completed;

    // Constructor: Recibe la conexión a la base de datos cuando se instancia el objeto
    public function __construct($db) {
        $this->conn = $db;
    }

    // 1. LEER TAREAS (READ)
    public function readAll() {
        $query = "SELECT * FROM " . $this->table_name . " ORDER BY created_at DESC";
        $stmt = this −> conn −> prepare(query);
        $stmt->execute();
        return $stmt;
    }

    // 2. CREAR TAREA (CREATE)
    public function create() {
        $query = "INSERT INTO " . $this->table_name . " (title) VALUES (:title)";
        $stmt = this −> conn −> prepare(query);

        // Limpiar entrada (evitar HTML/scripts dañinos)
        this −> title = htmlspecialchars(strip_tags(this->title));

        // Vincular parámetros para prevenir SQL Injection
        $stmt->bindParam(":title", $this->title);
```

```
        if($stmt->execute()) {
            return true;
        }
        return false;
    }

    // 3. CAMBIAR ESTADO / COMPLETAR (UPDATE)
    public function toggleComplete() {
        $query = "UPDATE " . $this->table_name . " SET completed = :completed WHERE id =
:id";
        $stmt = this −> conn −> prepare(query);

        $stmt->bindParam(":completed", $this->completed);
        $stmt->bindParam(":id", $this->id);

        return $stmt->execute();
    }

    // 4. ELIMINAR TAREA (DELETE)
    public function delete() {
        $query = "DELETE FROM " . $this->table_name . " WHERE id = :id";
        $stmt = this −> conn −> prepare(query);

        $stmt->bindParam(":id", $this->id);

        return $stmt->execute();
    }
}
```

index.php

```
<?php
require_once "Database.php";
require_once "Task.php";

// Instanciar la base de datos y obtener la conexión
$database = new Database();
$db = $database->getConnection();

// Instanciar el objeto Task
task = newTask(db);

// Procesar Acciones (POST / GET)
if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST["action"])) {
    if ($_POST["action"] == "add" && !empty($_POST["title"])) {
        $task->title = $_POST["title"];
```

```
        $task->create();
    }
}

if (isset($_GET["action"])) {
    if ($_GET["action"] == "toggle") {
        $task->id = $_GET["id"];
        $task->completed = $_GET["status"] == "1" ? 0 : 1;
        $task->toggleComplete();
    } elseif ($_GET["action"] == "delete") {
        $task->id = $_GET["id"];
        $task->delete();
    }
    // Redirigir para limpiar la URL
    header("Location: index.php");
    exit();
}

// Obtener todas las tareas para listarlas
$stmt = $task->readAll();
?>

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Gestor de Tareas POO</title>
    <style>
        :root {
            --primary: #4f46e5;
            --primary-hover: #4338ca;
            --bg-body: #f3f4f6;
            --bg-card: #ffffff;
            --text-main: #1f2937;
            --text-muted: #9ca3af;
            --success: #10b981;
            --danger: #ef4444;
            --danger-hover: #dc2626;
            --border: #e5e7eb;
            --radius: 12px;
            --shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1), 0 4px 6px -2px rgba(0, 0, 0, 0.05);
        }

        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0;
```

```
    }

    body {
        font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
Oxygen, Ubuntu, Cantarell, sans-serif;
        background-color: var(--bg-body);
        color: var(--text-main);
        display: flex;
        justify-content: center;
        align-items: center;
        min-height: 100vh;
        padding: 20px;
    }

    .container {
        background-color: var(--bg-card);
        width: 100%;
        max-width: 550px;
        padding: 32px;
        border-radius: var(--radius);
        box-shadow: var(--shadow);
    }

    header {
        margin-bottom: 24px;
    }

    header h1 {
        font-size: 1.6rem;
        font-weight: 700;
        color: var(--text-main);
    }

    header p {
        font-size: 0.9rem;
        color: var(--text-muted);
        margin-top: 4px;
    }

    /* Formulario */
    .task-form {
        display: flex;
        gap: 10px;
        margin-bottom: 24px;
    }

    .task-form input[type="text"] {
        flex: 1;
```

```
    padding: 12px 16px;
    border: 1px solid var(--border);
    border-radius: 8px;
    font-size: 0.95rem;
    outline: none;
    transition: border-color 0.2s, box-shadow 0.2s;
}

.task-form input[type="text"]:focus {
    border-color: var(--primary);
    box-shadow: 0 0 0 3px rgba(79, 70, 229, 0.2);
}

.btn-primary {
    background-color: var(--primary);
    color: white;
    border: none;
    padding: 12px 20px;
    border-radius: 8px;
    font-weight: 600;
    cursor: pointer;
    transition: background-color 0.2s;
}

.btn-primary:hover {
    background-color: var(--primary-hover);
}

/* Lista de tareas */
.task-list {
    list-style: none;
    display: flex;
    flex-direction: column;
    gap: 12px;
}

.task-item {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 14px 16px;
    background-color: #fafafa;
    border: 1px solid var(--border);
    border-radius: 8px;
    transition: border-color 0.2s, transform 0.1s;
}

.task-item:hover {
```

```
    border-color: #cbd5e1;
    transform: translateY(-1px);
}

.task-title {
    font-size: 0.95rem;
    font-weight: 500;
    word-break: break-word;
}

.task-item.is-completed .task-title {
    text-decoration: line-through;
    color: var(--text-muted);
}

/* Acciones / Botones */
.task-actions {
    display: flex;
    gap: 8px;
    align-items: center;
}

.btn-action {
    text-decoration: none;
    font-size: 0.85rem;
    font-weight: 600;
    padding: 6px 12px;
    border-radius: 6px;
    transition: all 0.2s;
}

.btn-toggle {
    background-color: #e0e7ff;
    color: var(--primary);
}

.btn-toggle:hover {
    background-color: #c7d2fe;
}

.btn-toggle.done {
    background-color: #d1fae5;
    color: #065f46;
}

.btn-toggle.done:hover {
    background-color: #a7f3d0;
}
```

```
        .btn-delete {
            background-color: #fee2e2;
            color: var(--danger);
        }

        .btn-delete:hover {
            background-color: #fca5a5;
            color: var(--danger-hover);
        }

        .empty-state {
            text-align: center;
            color: var(--text-muted);
            font-size: 0.9rem;
            padding: 20px 0;
        }
    </style>
</head>
<body>

    <div class="container">
        <header>
            <h1>Gestor de Tareas</h1>
            <p>Proyecto POO con PHP y MySQL</p>
        </header>

        <!-- Formulario para agregar tarea -->
        <form class="task-form" action="index.php" method="POST">
            <input type="hidden" name="action" value="add">
            <input type="text" name="title" placeholder="¿Qué tarea tienes pendiente?" required
autocomplete="off">
            <button type="submit" class="btn-primary">Agregar</button>
        </form>

        <!-- Listado de tareas -->
        <ul class="task-list">
            <?php
            $hasTasks = false;
            while ($row = $stmt->fetch(PDO::FETCH_ASSOC)):
                $hasTasks = true;
            ?>
                <li class="task-item <?= $row['completed'] ? 'is-completed' : '' ?>">
                    <span class="task-title">
                        <?= htmlspecialchars($row['title']) ?>
                    </span>
                    <div class="task-actions">
```

```
                    <a href="index.php?action=toggle&id=<?= $row['id'] ?>&status=<?=
$row['completed'] ?>"
                       class="btn-action btn-toggle <?= $row['completed'] ? 'done' : '' ?>">
                        <?= $row['completed'] ? 'Desmarcar' : 'Completar' ?>
                    </a>
                    <a href="index.php?action=delete&id=<?= $row['id'] ?>"
                       class="btn-action btn-delete"
                       onclick="return confirm('¿Seguro que deseas eliminar esta tarea?')">
                        Eliminar
                    </a>
                </div>
            </li>
        <?php endwhile; ?>

        <?php if (!$hasTasks): ?>
            <div class="empty-state">
                No hay tareas registradas aún. ¡Agrega la primera!
            </div>
        <?php endif; ?>
    </ul>
</div>

</body>
</html>
```

Explicación del código línea por línea:

Query.sql

Línea 1

SQL

```
CREATE DATABASE IF NOT EXISTS todo_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
```

- **CREATE DATABASE:** Es la instrucción de SQL que le ordena al servidor de MySQL crear un nuevo contenedor/espacio de almacenamiento (una base de datos).
- **IF NOT EXISTS:** Es una cláusula de seguridad. Le dice a MySQL: *"Crea esta base de datos solo si no hay otra que se llame igual"*. Evita que el script lance un error o sobrescriba datos si el script se ejecuta varias veces.
- **todo_db:** Es el nombre que le asignamos a la base de datos (abreviación de *To-Do DataBase*).
- **CHARACTER SET utf8mb4:** Define el juego de caracteres de la base de datos. Usar utf8mb4 garantiza soporte completo para caracteres internacionales, tildes, la letra "ñ" e incluso **emojis** (a diferencia del utf8 estándar que solo soporta hasta 3 bytes por carácter).

- **COLLATE utf8mb4_unicode_ci:** Define las reglas de comparación y ordenamiento del texto. La terminación _ci significa *Case Insensitive* (no distingue entre mayúsculas y minúsculas al hacer búsquedas o comparaciones).

Línea 2

SQL
USE todo_db;

- **USE todo_db;:** Le indica al cliente de MySQL que seleccione y active la base de datos todo_db. A partir de esta línea, cualquier tabla que se cree o consulta que se ejecute se aplicará dentro de este contenedor.

Línea 3

SQL
CREATE TABLE IF NOT EXISTS tasks (

- **CREATE TABLE:** Instrucción para crear una nueva tabla dentro de la base de datos activa.
- **IF NOT EXISTS:** Al igual que en la creación de la base de datos, evita que el código falle si la tabla tasks ya fue creada previamente.
- **tasks:** Es el nombre de la tabla (en plural por convención de buenas prácticas).

Línea 4

SQL
  id INT AUTO_INCREMENT PRIMARY KEY,

- **id:** Nombre de la primera columna.
- **INT:** Tipo de dato entero (números sin decimales).
- **AUTO_INCREMENT:** Le ordena a MySQL que genere el número automáticamente. Cada vez que se inserta una nueva tarea, MySQL suma 1 al ID anterior ($1, 2, 3, \dots$).
- **PRIMARY KEY:** Define esta columna como la **Llave Primaria**. Significa que este valor es el identificador único e irrepetible de cada fila en la tabla.

Línea 5

SQL
  title VARCHAR(255) NOT NULL,

- **title:** Nombre de la columna que almacenará el texto o descripción de la tarea.
- **VARCHAR(255):** Tipo de dato para cadenas de texto de longitud variable, con un límite máximo de 255 caracteres (suficiente espacio para el título de una tarea).
- **NOT NULL:** Restricción de integridad. Le indica a la base de datos que este campo **no puede quedarse vacío** al insertar una fila.

Línea 6

SQL

completed TINYINT(1) DEFAULT 0,

- **completed:** Nombre de la columna que indica el estado de la tarea (si está pendiente o completada).
- **TINYINT(1):** Tipo de dato entero muy pequeño (almacena un rango de -128 a 127). En MySQL es la forma estándar de manejar **valores booleanos** (0 representa false / pendiente, y 1 representa true / completada).
- **DEFAULT 0:** Valor por defecto. Si al insertar una tarea no especificamos su estado, la base de datos asumirá automáticamente que vale 0 (pendiente).

Línea 7

SQL
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

- **created_at:** Nombre de la columna para registrar el momento exacto en que se creó la tarea.
- **TIMESTAMP:** Tipo de dato de fecha y hora (almacena año, mes, día, hora, minutos y segundos).
- **DEFAULT CURRENT_TIMESTAMP:** Asigna automáticamente la fecha y hora del sistema en el instante preciso en que el registro se guarda en la base de datos, liberando al programador de enviar la fecha desde PHP.

Línea 8

SQL
);

- Cierra el paréntesis de definición de la tabla y el punto y coma (;) marca el final de la sentencia SQL.

💡 Resumen clave para la clase

Puedes decirle a tus alumnos que este script realiza tres acciones fundamentales:

1. **Crea el entorno de forma segura** (`CREATE DATABASE` con protección contra duplicados y soporte para emojis).
2. **Se posiciona en él** (`USE`).
3. **Crea la tabla contenedora** (`tasks`) con identificador autogenerado, validación para que la tarea no sea vacía, un estado booleano por defecto y auditoría de tiempo automática.

Database.php

Apertura y Definición de la Clase

PHP
<?php

- **Línea 1:** Apertura de la etiqueta de PHP. Le indica al servidor que el código a continuación debe ser interpretado por el motor de PHP.

PHP
class Database {

- **Línea 2:** Definición de la **Clase**. Usamos la palabra reservada `class` seguida del nombre `Database` (por convención, en mayúscula inicial y formato *CamelCase*). Esta clase funcionará como el "molde" o plantilla encargada de gestionar la conexión con MySQL.

Propiedades (Atributos) y Encapsulamiento

PHP
  private $host = "localhost";

- **Línea 4:** Declaración de la propiedad `$host` con visibilidad `private`. Indica el servidor donde reside la base de datos (en desarrollo local es `localhost` o `127.0.0.1`).
- **Concepto para los alumnos:** Al ser `private`, solo esta misma clase puede leer o modificar esta variable. Ningún código externo a la clase podrá alterarla.

PHP
  private $db_name = "todo_db";

- **Línea 5:** Propiedad privada que almacena el nombre exacto de la base de datos a la que nos conectaremos (`todo_db`).

PHP
  private $username = "root";

- **Línea 6:** Propiedad privada con el nombre del usuario de la base de datos (por defecto, en entornos locales como XAMPP/WAMP/LAMP se usa `root`).

PHP
  private $password = "";

- **Línea 7:** Propiedad privada con la contraseña del usuario de MySQL (por defecto en entornos locales suele ser una cadena vacía `""`).

PHP
  private $conn;

- **Línea 8:** Propiedad privada reservada para almacenar el **Objeto de Conexión** una vez que se establezca la comunicación con MySQL. Inicialmente no tiene valor asignado (`null`).

Métodos (Comportamientos): `getConnection()`

PHP
```
  public function getConnection() {
```

- **Línea 11:** Declaración del método `getConnection()`.
- **Concepto para los alumnos:** Se usa la palabra reservada `public` para que pueda ser invocado desde fuera de la clase (por ejemplo, desde `index.php` o `Task.php`). Este método es la "puerta de entrada" para obtener el recurso de conexión.

PHP
```
      $this->conn = null;
```

- **Línea 12:** Reinicializa la propiedad `$conn` asegurándose de que esté limpia antes de intentar conectar.
- **`$this`:** Palabra clave de POO que hace referencia al **objeto actual** (se lee como *"la propiedad conn de ESTA instancia"*).

Bloque de Conexión Segura: `try / catch` y PDO

PHP
```
      try {
```

- **Línea 14:** Inicia un bloque `try` (intentar). Se utiliza para envolver código propenso a fallar (como una conexión de red o base de datos). Si algo sale mal dentro de este bloque, la ejecución no detiene drásticamente la aplicación, sino que salta al bloque `catch`.

PHP
```
          $this->conn = new PDO(
              "mysql:host=" . $this->host . ";dbname=" . $this->db_name,
              $this->username,
              $this->password
          );
```

- **Líneas 16 a 20:** Instanciación de la extensión **PDO** (*PHP Data Objects*).
    - `new PDO(...)`: Crea un objeto nativo de PHP especializado en base de datos.

    - **DSN (Data Source Name):** `"mysql:host=" . $this->host . ";dbname=" . $this->db_name` especifica el motor (`mysql`), la dirección del servidor y el nombre de la base de datos concatenando las propiedades privadas mediante `$this`.
    - Pasa `$this->username` y `$this->password` como segundo y tercer argumento de autenticación.

PHP
```
        $this->conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

- **Línea 22:** Configura el comportamiento de PDO ante errores.
- **Concepto para los alumnos:** Le ordenamos a PDO que ante cualquier fallo SQL (error de sintaxis, tabla inexistente, etc.), **lance una Excepción** (`PDO::ERRMODE_EXCEPTION`). Esto facilita la detección de errores durante el desarrollo.

PHP
```
    } catch(PDOException $exception) {
```

- **Línea 23:** Cierra el bloque `try` e inicia el bloque `catch`. Si ocurrió algún error dentro del `try`, PHP captura el objeto del error y lo asigna a la variable `$exception` (que es de tipo `PDOException`).

PHP
```
        echo "Error de conexión: " . $exception->getMessage();
```

- **Línea 24:** Muestra un mensaje amigable en pantalla concatenando el mensaje técnico del error mediante `$exception->getMessage()`.

PHP
```
    }
```

- **Línea 25:** Cierre del bloque `catch`.

Retorno de la Conexión y Cierre de la Clase

```
PHP
        return $this->conn;
```

- **Línea 27:** Retorna el objeto de conexión `$this->conn` (ya sea la conexión activa o `null` si ocurrió un error) al código que haya llamado al método `getConnection()`.

```
PHP
    }
}
```

- **Líneas 28 y 29:** Cierre de la función `getConnection()` y cierre de la clase `Database`.

💡 Resumen didáctico para la clase

1. **Encapsulamiento:** Ocultamos los datos sensibles (credenciales) con `private`.
2. **Reutilización:** La clase `Database` se puede importar en cualquier proyecto PHP sin modificar una sola línea de su lógica.
3. **Manejo Seguro de Errores:** Usamos `PDO` junto con `try/catch` para prevenir brechas de seguridad y reaccionar limpiamente ante caídas del servidor.

Task.php

Apertura y Declaración de la Clase

```
PHP
<?php
```

- **Línea 1:** Abre la etiqueta PHP para que el servidor procese el código.

```
PHP
class Task {
```

- **Línea 2:** Declara la clase `Task`. Representa el "molde" o modelo de datos para cada tarea individual de nuestra aplicación.

Propiedades (Atributos) y Visibilidad

```
PHP
    private $conn;
```

- **Línea 4:** Propiedad privada para guardar el objeto de conexión a la base de datos (PDO). Al ser `private`, ningún código fuera de la clase puede acceder directamente a la conexión.

```php
   private $table_name = "tasks";
```

- **Línea 5:** Propiedad privada que almacena el nombre de la tabla en MySQL (`tasks`).
- **Concepto para alumnos:** Centralizar el nombre de la tabla en una variable permite que, si el nombre de la tabla cambia en la base de datos, solo debamos actualizarlo en esta línea y no en todas las consultas SQL de la clase.

```php
   public $id;
    public $title;
    public $completed;
```

- **Líneas 8 a 10:** Propiedades públicas que representan las columnas de la tabla:
  - `$id`: Identificador único de la tarea.
  - `$title`: Descripción o título de la tarea.
  - `$completed`: Estado (0 = pendiente, 1 = completada).
- **Concepto para alumnos:** Tienen visibilidad `public` para que desde la vista o el controlador (`index.php`) se puedan asignar o leer sus valores de forma directa (`$task->title = "Estudiar POO";`).

El Constructor e Inyección de Dependencias

```php
   public function __construct($db) {
```

- **Línea 13:** Declaración del **Constructor** (`__construct`). Es un método mágico en PHP que se ejecuta automáticamente de forma inmediata cuando instanciamos la clase con `new Task($db)`.

```php
       $this->conn = $db;
    }
```

- **Línea 14 y 15:** Asigna el objeto de conexión recibido en el parámetro `$db` a la propiedad interna `$this->conn`.
- **Concepto para alumnos (Inyección de Dependencias):** La clase `Task` no crea su propia conexión a la base de datos, sino que **la recibe** desde afuera. Esto hace al código más modular y evita abrir múltiples conexiones innecesarias en la base de datos.

1. Método de Lectura: `readAll()` (Read)

```
PHP
   public function readAll() {
```

- **Línea 18:** Método público encargado de consultar y obtener todas las tareas registradas en la base de datos.

```
PHP
       $query = "SELECT * FROM " . $this->table_name . " ORDER BY
created_at DESC";
```

- **Línea 19:** Define la consulta SQL concatenando la propiedad `$this->table_name`. `ORDER BY created_at DESC` ordena las tareas para que las más recientes aparezcan al principio.

```
PHP
       $stmt = $this->conn->prepare($query);
```

- **Línea 20:** Prepara la consulta SQL dentro de la conexión PDO y devuelve un objeto *Statement* (`$stmt`).

```
PHP
       $stmt->execute();
```

- **Línea 21:** Ejecuta la sentencia SQL en MySQL.

```
PHP
       return $stmt;
    }
```

- **Líneas 22 y 23:** Retorna el objeto `$stmt` con los resultados listos para ser recorridos en la vista mediante un bucle (`while`).

2. Método de Creación: `create()` (Create)

```
PHP
   public function create() {
```

- **Línea 26:** Método encargado de insertar una nueva tarea en la base de datos.

```
PHP
       $query = "INSERT INTO " . $this->table_name . " (title) VALUES
(:title)";
```

- **Línea 27:** Consulta SQL de inserción. Usa el marcador de posición `:title` en lugar de concatenar directamente la variable.

```
PHP
       $stmt = $this->conn->prepare($query);
```

- **Línea 28:** Prepara la sentencia SQL.

```
PHP
       $this->title = htmlspecialchars(strip_tags($this->title));
```

- **Línea 31 (Sanitización):**
  - `strip_tags()`: Elimina etiquetas HTML y PHP que el usuario intente ingresar.
  - `htmlspecialchars()`: Convierte caracteres especiales en entidades HTML (ejemplo: `<` se convierte en `<`).
  - **Concepto para alumnos:** Protege la aplicación contra ataques **XSS (Cross-Site Scripting)**.

```
PHP
       $stmt->bindParam(":title", $this->title);
```

- **Línea 34 (Seguridad SQL):** Vincula el marcador de posición `:title` con la propiedad `$this->title`.
- **Concepto para alumnos:** Evita ataques de **SQL Injection** al separar la estructura de la consulta de los datos enviados por el usuario.

```
PHP
       if($stmt->execute()) {
            return true;
        }
        return false;
    }
```

- **Líneas 36 a 40:** Ejecuta la consulta. Si MySQL inserta el registro con éxito retorna `true`, de lo contrario retorna `false`.

3. Método de Actualización: `toggleComplete()` (Update)

```
PHP
   public function toggleComplete() {
```

- **Línea 43:** Método encargado de alternar o cambiar el estado de la tarea (de pendiente a completado o viceversa).

```
PHP
       $query = "UPDATE " . $this->table_name . " SET completed =
:completed WHERE id = :id";
        $stmt = $this->conn->prepare($query);
```

- **Líneas 44 y 45:** Prepara la sentencia SQL `UPDATE` especificando qué registro se modificará con la cláusula `WHERE id = :id`.

```
PHP
       $stmt->bindParam(":completed", $this->completed);
        $stmt->bindParam(":id", $this->id);
```

- **Líneas 47 y 48:** Vincula los dos marcadores de posición (`:completed` e `:id`) con los valores almacenados en las propiedades del objeto (`$this->completed` y `$this->id`).

```
PHP
       return $stmt->execute();
    }
```

- **Líneas 50 y 51:** Ejecuta la consulta y retorna directamente el resultado booleano (`true` si actualizó o `false` si falló).

4. Método de Eliminación: `delete()` (Delete)

```
PHP
   public function delete() {
```

- **Línea 54:** Método encargado de borrar un registro de la base de datos.

```php
PHP
        $query = "DELETE FROM " . $this->table_name . " WHERE id = :id";
         $stmt = $this->conn->prepare($query);
```

- **Líneas 55 y 56:** Prepara la sentencia SQL `DELETE FROM tasks WHERE id = :id`.

```php
PHP
        $stmt->bindParam(":id", $this->id);
```

- **Línea 58:** Vincula el identificador de la tarea a eliminar (`:id`) con la propiedad `$this->id`.

```php
PHP
        return $stmt->execute();
    }
}
```

- **Líneas 60 a 63:** Ejecuta la eliminación en MySQL y retorna el resultado. Cierra la función y la definición de la clase `Task`.

💡 Resumen clave para explicar a la clase

1. **Atributos = Columnas de la tabla:** La clase tiene propiedades públicas que coinciden con los datos que queremos manipular (`id`, `title`, `completed`).
2. **Métodos = Operaciones CRUD:** Cada acción clave del sistema (`Create`, `Read`, `Update`, `Delete`) se convierte en un método de la clase.
3. **Seguridad Integrada:**
    - **Limpieza de código:** `strip_tags` + `htmlspecialchars` previenen código dañino en la interfaz.
    - **Prepared Statements:** `prepare()` + `bindParam()` evitan ataques de inyección SQL.

index.php

1. Importación de Clases y Preparación (PHP)

```php
PHP
require_once "Database.php";
```

```
require_once "Task.php";
```

- **Líneas 2-3:** Incluyen los archivos donde residen la clase de conexión a la base de datos y la clase modelo de la tarea (`Task`). Se usa `require_once` para asegurar que solo se carguen una vez y detener la ejecución si falta algún archivo crítico.

```
PHP
$database = new Database();
$db = $database->getConnection();
```

- **Línea 6:** Instancia un nuevo objeto de la clase `Database`.
- **Línea 7:** Llama al método `getConnection()` para obtener y guardar la conexión PDO activa en la variable `$db`.

```
PHP
$task = new Task($db);
```

- **Línea 10:** Crea el objeto `$task` pasando la conexión `$db` al constructor de la clase `Task` (inyección de dependencias).

2. Procesamiento de Acciones backend (Crear, Cambiar Estado, Eliminar)

```
PHP
if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST["action"])) {
    if ($_POST["action"] == "add" && !empty($_POST["title"])) {
        $task->title = $_POST["title"];
        $task->create();
    }
}
```

- **Línea 13:** Comprueba si el formulario fue enviado mediante el método HTTP **POST** y si existe el campo oculto `action`.
- **Línea 14:** Valida que la acción sea `"add"` y que el campo `title` no viaje vacío.
- **Línea 15:** Asigna el título ingresado por el usuario a la propiedad `$title` del objeto `$task`.
- **Línea 16:** Ejecuta el método `create()` del modelo para insertar la nueva tarea en MySQL.

```php
if (isset($_GET["action"])) {
    if ($_GET["action"] == "toggle") {
        $task->id = $_GET["id"];
        $task->completed = $_GET["status"] == "1" ? 0 : 1;
        $task->toggleComplete();
```

- **Línea 20:** Comprueba si hay una acción solicitada mediante la URL (método **GET**).
- **Línea 21:** Si la acción es `"toggle"` (cambiar estado):
    - **Línea 22:** Asigna el ID de la tarea desde la URL.
    - **Línea 23:** Aplica un **operador ternario**: si la tarea actual figura como completada (`"1"`), invierte el estado a `0` (pendiente), y viceversa.
    - **Línea 24:** Llama al método `toggleComplete()` para actualizar el registro en la base de datos.

```php
   } elseif ($_GET["action"] == "delete") {
        $task->id = $_GET["id"];
        $task->delete();
    }
```

- **Líneas 25-27:** Si la acción solicitada por URL es `"delete"`, asigna el ID recibido y ejecuta el método `delete()` de la clase `Task`.

```php
   header("Location: index.php");
    exit();
}
```

- **Línea 29:** Redirige al navegador de vuelta a `index.php` mediante el encabezado HTTP. Esto aplica el patrón **Post/Redirect/Get (PRG)** para limpiar los parámetros de la URL y evitar que el usuario reenvíe la acción al recargar la página (`F5`).
- **Línea 30:** Detiene inmediatamente la ejecución del script tras enviar la orden de redirección.

3. Consulta de Datos para la Vista

```php
$stmt = $task->readAll();
```

- **Línea 34:** Llama al método `readAll()` del objeto `$task`, el cual retorna un objeto `PDOStatement` (resultado de la consulta `SELECT * FROM tasks`) listo para ser recorrido en la plantilla HTML.

4. Estructura HTML y Estilos CSS

```
HTML
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Gestor de Tareas POO</title>
```

- **Líneas 37-42:** Encabezado estándar de HTML5 con codificación UTF-8 para caracteres en español y soporte responsivo para dispositivos móviles.

```
CSS
<style>
    :root { ... }
    * { ... }
    body { ... }
    .container { ... }
    ...
</style>
```

- **Líneas 43-186:** Bloque de estilos CSS embebidos que define:
    - **Variables CSS (`:root`):** Paleta de colores, bordes redondeados y sombras para facilitar el mantenimiento visual.
    - **Layout flexbox:** Alineación centrada para la tarjeta contenedora en pantalla completa.
    - **Diseño UI:** Estilos para el formulario, la lista de items, estados completados (texto tachado `line-through`), botones con pseudo-clases `:hover` y aviso de estado vacío.

5. Cuerpo de la Aplicación y Formulario

```
HTML
<body>
    <div class="container">
        <header>
            <h1>Gestor de Tareas</h1>
```

```
            <p>Proyecto POO con PHP y MySQL</p>
        </header>
```

- **Líneas 188-195:** Envoltorio principal y encabezado visual del módulo.

```
HTML
        <form class="task-form" action="index.php" method="POST">
            <input type="hidden" name="action" value="add">
            <input type="text" name="title" placeholder="¿Qué tarea
tienes pendiente?" required autocomplete="off">
            <button type="submit" class="btn-primary">Agregar</button>
        </form>
```

- **Línea 198:** Formulario que envía sus datos mediante `POST` hacia el mismo archivo `index.php`.
- **Línea 199:** Input oculto (`type="hidden"`) que le indica al controlador que la acción a realizar es `"add"`.
- **Líneas 200-201:** Input de texto obligatorio (`required`) para la descripción y botón de envío.

6. Renderizado Dinámico de la Lista de Tareas

```
PHP
        <ul class="task-list">
            <?php
            $hasTasks = false;
            while ($row = $stmt->fetch(PDO::FETCH_ASSOC)):
                $hasTasks = true;
            ?>
```

- **Línea 206:** Inicializa la bandera `$hasTasks` en `false` para rastrear si existen registros.
- **Línea 207:** Inicia un bucle `while` usando `fetch(PDO::FETCH_ASSOC)` para recorrer cada fila devuelta por la consulta base de datos como un arreglo asociativo.
- **Línea 208:** Cambia la bandera a `true` al confirmar la presencia de al menos un elemento.

```
HTML
                <li class="task-item <?= $row['completed'] ?
'is-completed' : '' ?>">
                    <span class="task-title">
```

```
                        <?= htmlspecialchars($row['title']) ?>
                    </span>
```

- **Línea 210:** Evalúa en línea si la columna `completed` es verdadera (1). De ser así, añade la clase CSS `is-completed` para tachar el texto.
- **Línea 212:** Muestra el título de la tarea aplicando `htmlspecialchars()` para prevenir ataques de vulnerabilidad XSS (Cross-Site Scripting).

```
HTML
                    <div class="task-actions">
                        <a href="index.php?action=toggle&id=<?=
$row['id'] ?>&status=<?= $row['completed'] ?>"
                           class="btn-action btn-toggle <?=
$row['completed'] ? 'done' : '' ?>">
                            <?= $row['completed'] ? 'Desmarcar' :
'Completar' ?>
                        </a>
```

- **Líneas 215-218:** Genera un enlace **GET** de acción condicional para marcar o desmarcar la tarea. Modifica tanto la URL enviada (`action=toggle`), la clase dinámica (`done`) y el texto visible (*Desmarcar* o *Completar*) según el valor de `$row['completed']`.

```
HTML
                    <a href="index.php?action=delete&id=<?=
$row['id'] ?>"
                       class="btn-action btn-delete"
                       onclick="return confirm('¿Seguro que deseas
eliminar esta tarea?')">
                        Eliminar
                    </a>
                </div>
            </li>
        <?php endwhile; ?>
```

- **Líneas 219-224:** Enlace de eliminación vía **GET** que dispara una alerta JavaScript (`confirm()`) en el cliente antes de procesar el enlace.
- **Línea 225:** Cierra el bucle de iteración `while`.

```
PHP
        <?php if (!$hasTasks): ?>
```

```
                <div class="empty-state">
                    No hay tareas registradas aún. ¡Agrega la primera!
                </div>
            <?php endif; ?>
        </ul>
    </div>
</body>
</html>
```

- **Líneas 227-231:** Si la variable `$hasTasks` permaneció en `false` (la tabla de la BD estaba vacía), muestra un mensaje informativo de estado vacío.
- **Líneas 232-234:** Cierre de las etiquetas estructurales de HTML.

Conceptos clave recomendados para destacar a los alumnos:

1. **Inyección de Dependencias:** Pasar la conexión PDO `$db` al instanciar $task = newTask(\text{db})$.
2. **Seguridad Básica:** Uso de `htmlspecialchars()` en la vista contra XSS.
3. **Patrón PRG (Post/Redirect/Get):** El uso de `header("Location: ...")` para evitar duplicación de registros al refrescar el navegador.

Otros proyectos a desarrollar:

1. Catálogo / Inventario de Productos (Nivel: Principiante - Medio)

Un clásico del desarrollo web que amplía la lógica del *To-Do List*.

- **¿De qué trata?:** Sistema para registrar, editar y listar productos (por ejemplo, una tienda de tecnología o libros) con nombre, precio, stock y categoría.
- **Nuevos conceptos de POO / PHP a enseñar:**
  - **Manejo de archivos (Uploads):** Guardar imágenes de productos en el servidor y su ruta en la base de datos.
  - **Métodos con filtros:** Crear métodos en la clase `Product` como `readByCategory($category_id)` o `searchByTitle($keyword)`.
  - **Relación de Tablas en MySQL:** Unir la tabla `products` con `categories` mediante una clave foránea (`FOREIGN KEY`).

2. Sistema de Blog / Artículos con Comentarios (Nivel: Medio)

- **¿De qué trata?:** Plataforma donde se publican artículos y los visitantes pueden dejar comentarios.
- **Nuevos conceptos de POO / PHP a enseñar:**

    - **Múltiples Clases Interactuando:** Crear la clase `Post` y la clase `Comment`.
    - **Composición / Asociación:** Explicar cómo un objeto `Post` contiene o se relaciona con múltiples objetos `Comment`.
    - **Manejo de Estados/Publicación:** Campos booleanos o cadenas para marcar artículos como "Borrador" o "Publicado".

3. Sistema de Registro y Control de Asistencia (Nivel: Medio)

Un proyecto muy cercano al entorno educativo de los alumnos.

- **¿De qué trata?:** Permite registrar estudiantes y marcar su asistencia diaria (Presente, Ausente, Justificado) por fecha.
- **Nuevos conceptos de POO / PHP a enseñar:**
    - **Manejo de Fechas en PHP y MySQL:** Trabajar con la clase nativa `DateTime` de PHP y campos `DATE` en MySQL.
    - **Inserciones Múltiples:** Procesar arreglos de datos en un solo formulario (marcar la asistencia de varios estudiantes a la vez).
    - **Métodos Estadísticos:** Crear un método en la clase `Student` llamado `getAttendancePercentage()` para calcular el porcentaje de asistencia.

4. Gestor de Contactos / Agenda Telefónica (Nivel: Principiante)

- **¿De qué trata?:** Una libreta digital para guardar contactos con nombre, teléfono, correo, dirección y etiquetas (Familia, Trabajo, Amigos).
- **Nuevos conceptos de POO / PHP a enseñar:**
    - **Búsquedas Dinámicas en MySQL:** Uso de `LIKE %busqueda%` en las consultas preparadas.
    - **Validación de Datos:** Uso de filtros nativos de PHP (`filter_var($email, FILTER_VALIDATE_EMAIL)`) dentro del modelo antes de guardar.

5. Sistema Básico de Autenticación y Roles de Usuario (Nivel: Avanzado)

El paso definitivo para que entiendan cómo se protegen las aplicaciones web reales.

- **¿De qué trata?:** Un módulo de Login y Registro donde los usuarios se autentican y acceden a un panel privado según su rol (Administrador vs. Usuario estándar).
- **Nuevos conceptos de POO / PHP a enseñar:**
    - **Seguridad y Encriptación:** Uso de `password_hash()` y `password_verify()` en la clase `User`.
    - **Manejo de Sesiones:** Uso de `session_start()` e integración con métodos como `User::login()` y `User::isLoggedIn()`.
    - **Encapsulamiento de Sesión:** Crear una clase dedicada llamada `SessionManager` para controlar el estado del usuario en toda la aplicación.

---

4. Gestor de Contactos / Agenda Telefónica (Nivel: Principiante)

- **¿De qué trata?:** Una libreta digital para guardar contactos con nombre, teléfono, correo, dirección y etiquetas (Familia, Trabajo, Amigos).
- **Nuevos conceptos de POO / PHP a enseñar:**
    - **Búsquedas Dinámicas en MySQL:** Uso de `LIKE %busqueda%` en las consultas preparadas.
    - **Validación de Datos:** Uso de filtros nativos de PHP (`filter_var($email, FILTER_VALIDATE_EMAIL)`) dentro del modelo antes de guardar.

Arquitectura del Proyecto

Mantendremos una estructura de **3 archivos principales** y un script SQL:

```
agenda_poo/
├── schema.sql      # Script de la base de datos
├── Database.php    # Clase para la conexión PDO
├── Contact.php     # Clase del Modelo / Lógica de Contactos
├── index.php       # Controlador y Vista (HTML + CSS)
└── styles.css      # Controlador y Vista (HTML + CSS)
```

schema.sql

```
CREATE DATABASE IF NOT EXISTS agenda_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE agenda_db;

CREATE TABLE IF NOT EXISTS contacts (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    email VARCHAR(100) NULL,
    category ENUM('Familia', 'Trabajo', 'Amigos', 'Otros') DEFAULT 'Otros',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Database.php

```
<?php
class Database {
    private $host = "localhost";
    private $db_name = "agenda_db";
    private $username = "root";
```

```
    private $password = "";
    private $conn;

    public function getConnection() {
        $this->conn = null;

        try {
            $this->conn = new PDO(
                "mysql:host=" . $this->host . ";dbname=" . $this->db_name,
                $this->username,
                $this->password
            );
            $this->conn->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);
        } catch(PDOException $exception) {
            echo "Error de conexión: " . $exception->getMessage();
        }

        return $this->conn;
    }
}
```

Contact.php

```
<?php
class Contact {
    private $conn;
    private $table_name = "contacts";

    // Propiedades del Objeto
    public $id;
    public $name;
    public $phone;
    public $email;
    public $category;

    public function __construct($db) {
        $this->conn = $db;
    }

    // 1. LEER TODOS O BUSCAR
    public function read($search = "") {
        if (!empty($search)) {
            // Consulta de búsqueda con operador LIKE
            $query = "SELECT * FROM " . $this->table_name . "
                    WHERE name LIKE :search OR phone LIKE :search OR email LIKE :search
                    ORDER BY name ASC";
            $stmt = this -> conn -> prepare(query);
```

```
            $search_term = "%{$search}%";
            $stmt->bindParam(":search", $search_term);
        } else {
            $query = "SELECT * FROM " . $this->table_name . " ORDER BY name ASC";
            $stmt = this −> conn −> prepare(query);
        }

        $stmt->execute();
        return $stmt;
    }

    // 2. CREAR CONTACTO
    public function create() {
        $query = "INSERT INTO " . $this->table_name . " (name, phone, email, category)
                VALUES (:name, :phone, :email, :category)";
        $stmt = this −> conn −> prepare(query);

        // Sanitización de entradas
        this −> name = htmlspecialchars(strip_tags(this->name));
        this −> phone = htmlspecialchars(strip_tags(this->phone));
        this −> email = htmlspecialchars(strip_tags(this->email));
        this −> category = htmlspecialchars(strip_tags(this->category));

        // Vinculación de parámetros
        $stmt->bindParam(":name", $this->name);
        $stmt->bindParam(":phone", $this->phone);
        $stmt->bindParam(":email", $this->email);
        $stmt->bindParam(":category", $this->category);

        return $stmt->execute();
    }

    // 3. ELIMINAR CONTACTO
    public function delete() {
        $query = "DELETE FROM " . $this->table_name . " WHERE id = :id";
        $stmt = this −> conn −> prepare(query);
        $stmt->bindParam(":id", $this->id);
        return $stmt->execute();
    }
}
```

index.php

```
<?php
require_once "Database.php";
require_once "Contact.php";
```

```php
$database = new Database();
$db = $database->getConnection();
$contact = new Contact($db);

// Mensaje de estado para feedback al usuario
$message = "";
$messageType = "";

// 1. Procesar creación de contacto (POST)
if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST["action"]) &&
$_POST["action"] == "add") {
    if (!empty($_POST["name"]) && !empty($_POST["phone"])) {
        $contact->name = $_POST["name"];
        $contact->phone = $_POST["phone"];
        $contact->email = $_POST["email"] ?? "";
        $contact->category = $_POST["category"] ?? "Otros";

        if ($contact->create()) {
            $message = "¡Contacto guardado exitosamente!";
            $messageType = "success";
        } else {
            $message = "Ocurrió un error al guardar el contacto.";
            $messageType = "error";
        }
    }
}

// 2. Procesar eliminación (GET)
if (isset($_GET["action"]) && $_GET["action"] == "delete" && isset($_GET["id"])) {
    $contact->id = $_GET["id"];
    $contact->delete();
    header("Location: index.php");
    exit();
}

// 3. Capturar término de búsqueda si existe
$search_query = $_GET["search"] ?? "";
$stmt = $contact->read($search_query);
?>

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Agenda de Contactos POO</title>
  <link rel="stylesheet" href="styles.css">
```

```
  <!-- Fuente Google Fonts: Plus Jakarta Sans -->
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link
href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700
&display=swap" rel="stylesheet">
</head>
<body>

  <!-- Navegación -->
  <header class="navbar">
    <div class="logo">
      <span class="logo-icon">📇</span> ContactApp <span
class="badge-mini">POO</span>
    </div>
    <div class="nav-info">
      <span class="status-dot"></span> MySQL Conectado
    </div>
  </header>

  <!-- Hero Section -->
  <main class="container">
    <section class="hero">
      <span class="badge">✨ Proyecto Didáctico PHP & MySQL</span>
      <h1>Gestión de Contactos <span class="gradient-text">Moderna</span></h1>
      <p class="hero-subtitle">
        Administra tu agenda telefónica aplicando Programación Orientada a Objetos y
consultas seguras con PDO.
      </p>
    </section>

    <!-- Alertas de feedback -->
    <?php if(!empty($message)): ?>
      <div class="alert alert-<?= $messageType ?>">
        <?= htmlspecialchars($message) ?>
      </div>
    <?php endif; ?>

    <!-- Cuadrícula Principal (Formulario + Lista) -->
    <div class="app-grid">

      <!-- Columna Izquierda: Formulario -->
      <div class="card card-form">
        <div class="card-header">
          <h2>👤 Nuevo Contacto</h2>
          <p>Llena los datos para guardar en MySQL</p>
        </div>
```

```
        <form action="index.php" method="POST" class="contact-form">
          <input type="hidden" name="action" value="add">

          <div class="form-group">
            <label for="name">Nombre Completo *</label>
            <input type="text" id="name" name="name" required placeholder="Ej. Ana Gómez"
autocomplete="off">
          </div>

          <div class="form-group">
            <label for="phone">Teléfono *</label>
            <input type="tel" id="phone" name="phone" required placeholder="Ej. +58 412
1234567" autocomplete="off">
          </div>

          <div class="form-group">
            <label for="email">Correo Electrónico</label>
            <input type="email" id="email" name="email" placeholder="ejemplo@correo.com"
autocomplete="off">
          </div>

          <div class="form-group">
            <label for="category">Categoría</label>
            <select id="category" name="category">
              <option value="Familia">Familia</option>
              <option value="Trabajo">Trabajo</option>
              <option value="Amigos">Amigos</option>
              <option value="Otros" selected>Otros</option>
            </select>
          </div>

          <button type="submit" class="btn btn-primary btn-block">Guardar Contacto</button>
        </form>
      </div>

      <!-- Columna Derecha: Búsqueda y Lista -->
      <div class="card card-list">
        <div class="card-header flex-header">
          <div>
            <h2>📋 Mis Contactos</h2>
            <p>Lista almacenada en la base de datos</p>
          </div>
        </div>

        <!-- Buscador -->
        <form action="index.php" method="GET" class="search-bar">
          <input type="text" name="search" placeholder="Buscar por nombre, teléfono o
email..." value="<?= htmlspecialchars($search_query) ?>" autocomplete="off">
```

```
    <button type="submit" class="btn btn-secondary">Buscar</button>
    <?php if(!empty($search_query)): ?>
      <a href="index.php" class="btn btn-icon" title="Limpiar búsqueda">✕</a>
    <?php endif; ?>
  </form>

  <!-- Lista de Contactos -->
  <ul class="contact-list">
    <?php
    $hasContacts = false;
    while ($row = $stmt->fetch(PDO::FETCH_ASSOC)):
      $hasContacts = true;
    ?>
      <li class="contact-item">
        <div class="contact-avatar">
          <?= strtoupper(substr($row['name'], 0, 1)) ?>
        </div>
        <div class="contact-info">
          <div class="contact-title">
            <h3><?= htmlspecialchars($row['name']) ?></h3>
            <span class="badge-cat badge-<?= strtolower($row['category']) ?>">
              <?= htmlspecialchars($row['category']) ?>
            </span>
          </div>
          <div class="contact-details">
            <span>📞 <?= htmlspecialchars($row['phone']) ?></span>
            <?php if(!empty($row['email'])): ?>
              <span>✉️ <?= htmlspecialchars($row['email']) ?></span>
            <?php endif; ?>
          </div>
        </div>
        <a href="index.php?action=delete&id=<?= $row['id'] ?>"
          class="btn-delete"
          onclick="return confirm('¿Seguro que deseas eliminar este contacto?')"
          title="Eliminar contacto">
          🗑️
        </a>
      </li>
    <?php endwhile; ?>

    <?php if (!$hasContacts): ?>
      <div class="empty-state">
        <span class="empty-icon">🔍</span>
        <p>No se encontraron contactos en la agenda.</p>
      </div>
    <?php endif; ?>
  </ul>
</div>
```

```
    </div>
  </main>

</body>
</html>
```

styles.css

```
/*
================================================================================
   1. VARIABLES DE DISEÑO (Design Tokens)

================================================================================ */
:root {
  --font-family: 'Plus Jakarta Sans', -apple-system, BlinkMacSystemFont, sans-serif;

  /* Colores */
  --bg-main: #0b0f19;
  --bg-card: rgba(22, 31, 48, 0.7);
  --bg-card-hover: rgba(30, 41, 64, 0.85);
  --border-color: rgba(255, 255, 255, 0.08);
  --border-color-hover: rgba(99, 102, 241, 0.5);

  --primary: #6366f1;
  --primary-hover: #4f46e5;
  --primary-glow: rgba(99, 102, 241, 0.35);

  --text-main: #f8fafc;
  --text-muted: #94a3b8;

  --success: #10b981;
  --danger: #ef4444;

  /* Radios y Sombras */
  --radius-sm: 8px;
  --radius-md: 16px;
  --radius-lg: 24px;
  --shadow-main: 0 20px 40px -15px rgba(0, 0, 0, 0.5);
  --transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
}

/*
================================================================================
```

```
   2. RESETEO Y BASE

================================================================================ */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: var(--font-family);
  background-color: var(--bg-main);
  color: var(--text-main);
  min-height: 100vh;
  line-height: 1.6;
  overflow-x: hidden;
  background-image:
    radial-gradient(at 0% 0%, rgba(99, 102, 241, 0.15) 0px, transparent 50%),
    radial-gradient(at 100% 100%, rgba(168, 85, 247, 0.15) 0px, transparent 50%);
}

.container {
  max-width: 1100px;
  margin: 0 auto;
  padding: 0 24px 60px;
}

/*
================================================================================
   3. BARRA DE NAVEGACIÓN

================================================================================ */
.navbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 16px 40px;
  backdrop-filter: blur(12px);
  background: rgba(11, 15, 25, 0.8);
  border-bottom: 1px solid var(--border-color);
  position: sticky;
  top: 0;
  z-index: 100;
}
```

```
.logo {
  font-size: 1.25rem;
  font-weight: 700;
  display: flex;
  align-items: center;
  gap: 8px;
}

.badge-mini {
  font-size: 0.7rem;
  background: var(--primary);
  color: #fff;
  padding: 2px 6px;
  border-radius: 4px;
}

.nav-info {
  font-size: 0.85rem;
  color: var(--text-muted);
  display: flex;
  align-items: center;
  gap: 8px;
}

.status-dot {
  width: 8px;
  height: 8px;
  background-color: var(--success);
  border-radius: 50%;
  display: inline-block;
}

/*
================================================================================
   4. HERO SECTION Y ALERTAS

================================================================================ */
.hero {
  text-align: center;
  padding: 40px 20px 30px;
  max-width: 700px;
  margin: 0 auto;
}

.badge {
  display: inline-block;
```

```
  padding: 6px 16px;
  border-radius: 50px;
  background: rgba(99, 102, 241, 0.1);
  border: 1px solid rgba(99, 102, 241, 0.3);
  color: #818cf8;
  font-size: 0.85rem;
  font-weight: 600;
  margin-bottom: 16px;
}

.hero h1 {
  font-size: 2.5rem;
  font-weight: 700;
  line-height: 1.2;
  margin-bottom: 12px;
}

.gradient-text {
  background: linear-gradient(135deg, #818cf8 0%, #c084fc 100%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
}

.hero-subtitle {
  font-size: 1rem;
  color: var(--text-muted);
}

.alert {
  padding: 12px 20px;
  border-radius: var(--radius-sm);
  margin-bottom: 24px;
  text-align: center;
  font-weight: 500;
}
.alert-success { background: rgba(16, 185, 129, 0.15); border: 1px solid var(--success);
color: #6ee7b7; }
.alert-error { background: rgba(239, 68, 68, 0.15); border: 1px solid var(--danger); color:
#fca5a5; }

/*
==========================================================================
====
   5. LAYOUT DE APLICACIÓN (Grid)

==========================================================================
==== */
.app-grid {
```

```
  display: grid;
  grid-template-columns: 1fr 1.5fr;
  gap: 24px;
  align-items: start;
}

.card {
  background: var(--bg-card);
  backdrop-filter: blur(16px);
  border: 1px solid var(--border-color);
  border-radius: var(--radius-md);
  padding: 28px;
  box-shadow: var(--shadow-main);
}

.card-header {
  margin-bottom: 20px;
}

.card-header h2 {
  font-size: 1.2rem;
  font-weight: 600;
}

.card-header p {
  font-size: 0.85rem;
  color: var(--text-muted);
}

/*
==========================================================================
====
   6. FORMULARIO E INPUTS

==========================================================================
==== */
.contact-form {
  display: flex;
  flex-direction: column;
  gap: 16px;
}

.form-group {
  display: flex;
  flex-direction: column;
  gap: 6px;
}
```

```
.form-group label {
  font-size: 0.85rem;
  font-weight: 600;
  color: var(--text-muted);
}

input, select {
  font-family: inherit;
  background: rgba(11, 15, 25, 0.6);
  border: 1px solid var(--border-color);
  color: var(--text-main);
  padding: 10px 14px;
  border-radius: var(--radius-sm);
  font-size: 0.9rem;
  outline: none;
  transition: var(--transition);
}

input:focus, select:focus {
  border-color: var(--primary);
  box-shadow: 0 0 0 3px rgba(99, 102, 241, 0.2);
}

select option {
  background-color: var(--bg-main);
  color: var(--text-main);
}

/* Botones */
.btn {
  font-family: inherit;
  font-weight: 600;
  border-radius: var(--radius-sm);
  padding: 10px 18px;
  border: none;
  cursor: pointer;
  transition: var(--transition);
  display: inline-flex;
  align-items: center;
  justify-content: center;
  text-decoration: none;
}

.btn-primary {
  background-color: var(--primary);
  color: #ffffff;
  box-shadow: 0 4px 14px var(--primary-glow);
}
```

```
.btn-primary:hover {
  background-color: var(--primary-hover);
}

.btn-secondary {
  background: rgba(255, 255, 255, 0.08);
  color: var(--text-main);
  border: 1px solid var(--border-color);
}

.btn-secondary:hover {
  background: rgba(255, 255, 255, 0.15);
}

.btn-block { width: 100%; margin-top: 8px; }
.btn-icon { background: transparent; color: var(--text-muted); border: 1px solid
var(--border-color); padding: 10px 14px; }
.btn-icon:hover { color: var(--text-main); background: rgba(255,255,255,0.1); }

/*
==========================================================================
====
   7. LISTA DE CONTACTOS Y BUSCADOR

==========================================================================
==== */
.search-bar {
  display: flex;
  gap: 8px;
  margin-bottom: 20px;
}

.search-bar input {
  flex: 1;
}

.contact-list {
  list-style: none;
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.contact-item {
  background: rgba(11, 15, 25, 0.4);
  border: 1px solid var(--border-color);
  border-radius: var(--radius-sm);
```

```
  padding: 14px 16px;
  display: flex;
  align-items: center;
  gap: 14px;
  transition: var(--transition);
}

.contact-item:hover {
  border-color: var(--border-color-hover);
  background: var(--bg-card-hover);
}

.contact-avatar {
  width: 40px;
  height: 40px;
  border-radius: 50%;
  background: linear-gradient(135deg, var(--primary), #a855f7);
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: 700;
  font-size: 1.1rem;
  color: white;
  flex-shrink: 0;
}

.contact-info { flex: 1; }

.contact-title {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 2px;
}

.contact-title h3 {
  font-size: 0.95rem;
  font-weight: 600;
}

.badge-cat {
  font-size: 0.7rem;
  padding: 2px 8px;
  border-radius: 12px;
  font-weight: 600;
  background: rgba(255, 255, 255, 0.1);
  color: var(--text-muted);
}
```

```
.contact-details {
  font-size: 0.8rem;
  color: var(--text-muted);
  display: flex;
  gap: 12px;
  flex-wrap: wrap;
}

.btn-delete {
  text-decoration: none;
  font-size: 1rem;
  padding: 6px 10px;
  border-radius: 6px;
  transition: var(--transition);
  opacity: 0.7;
}

.btn-delete:hover {
  opacity: 1;
  background: rgba(239, 68, 68, 0.2);
}

.empty-state {
  text-align: center;
  padding: 40px 0;
  color: var(--text-muted);
}

.empty-icon {
  font-size: 2rem;
  display: block;
  margin-bottom: 8px;
}

/* Responsive */
@media (max-width: 850px) {
  .app-grid {
    grid-template-columns: 1fr;
  }
}
```

Explicación del código línea por línea:

schema.sql

Línea 1

```
SQL
CREATE DATABASE IF NOT EXISTS agenda_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
```

- **`CREATE DATABASE`:** Es la instrucción de SQL que le ordena a MySQL crear una nueva base de datos.
- **`IF NOT EXISTS`:** Una regla de protección. Le dice a MySQL: *"Crea esta base de datos solo si no existe una con el mismo nombre"*. Esto evita que el código lance un error si vuelves a ejecutar el script.
- **`agenda_db`:** El nombre de la base de datos.
- **`CHARACTER SET utf8mb4`:** Define el juego de caracteres. Usamos `utf8mb4` para que la base de datos pueda guardar correctamente tildes, la letra "ñ" y hasta **emojis** en los datos.
- **`COLLATE utf8mb4_unicode_ci`:** Define las reglas de ordenamiento y comparación de texto. La terminación `_ci` (*Case Insensitive*) indica que MySQL no diferenciará entre mayúsculas y minúsculas al buscar o comparar datos.

Línea 2

```
SQL
USE agenda_db;
```

- **`USE agenda_db;`:** Selecciona y activa la base de datos `agenda_db`. Le indica a MySQL que todas las tablas que creemos o consultas que ejecutemos a partir de este punto se harán dentro de esta base de datos.

Línea 3

```
SQL
CREATE TABLE IF NOT EXISTS contacts (
```

- **`CREATE TABLE`:** Instrucción para crear una nueva tabla.
- **`IF NOT EXISTS`:** Al igual que con la base de datos, asegura que no se genere un error si la tabla ya fue creada anteriormente.
- **`contacts`:** Nombre de la tabla en plural (*contactos*), siguiendo las buenas prácticas de nombrado en SQL.

Línea 4

```
SQL
   id INT AUTO_INCREMENT PRIMARY KEY,
```

- **`id`:** Nombre de la primera columna.
- **`INT`:** Tipo de dato entero (números sin decimales).
- **`AUTO_INCREMENT`:** MySQL le asignará automáticamente un número correlativo a cada nuevo contacto (1,2,3,…). El usuario o programador no necesita enviarlo.
- **`PRIMARY KEY`:** Marca este campo como la **Llave Primaria**, lo que garantiza que cada registro tenga un identificador único e irrepetible.

Línea 5

```
SQL
   name VARCHAR(100) NOT NULL,
```

- **`name`:** Nombre de la columna para almacenar el nombre del contacto.
- **`VARCHAR(100)`:** Tipo de dato para texto de longitud variable, con un límite de hasta 100 caracteres.
- **`NOT NULL`:** Restricción de validación. Indica que el nombre **es obligatorio**; la base de datos rechazará el registro si este campo se envía vacío.

Línea 6

```
SQL
   phone VARCHAR(20) NOT NULL,
```

- **`phone`:** Nombre de la columna para el número telefónico.
- **`VARCHAR(20)`:** Se usa texto (`VARCHAR`) en lugar de números (`INT`) porque los números telefónicos pueden incluir caracteres como el signo más (`+`), espacios o guiones (ej. `+58 412-1234567`).
- **`NOT NULL`:** Al igual que el nombre, es un campo obligatorio.

Línea 7

```
SQL
   email VARCHAR(100) NULL,
```

- **`email`:** Columna para guardar el correo electrónico del contacto.
- **`VARCHAR(100)`:** Espacio de texto de hasta 100 caracteres.
- **`NULL`:** Restricción opcional. Significa que el campo **puede quedarse vacío** si el usuario no tiene o no desea ingresar un correo.

Línea 8

```SQL
   category ENUM('Familia', 'Trabajo', 'Amigos', 'Otros') DEFAULT
'Otros',
```

- **`category`:** Nombre de la columna para la categoría del contacto.
- **`ENUM('Familia', 'Trabajo', 'Amigos', 'Otros')`:** Tipo de dato de lista predefinida (*Enumeración*). Limita los datos ingresados exclusivamente a una de las opciones especificadas dentro de las comillas.
- **`DEFAULT 'Otros'`:** Valor por defecto. Si al insertar un contacto no se especifica una categoría, MySQL le asignará automáticamente la etiqueta `'Otros'`.

Línea 9

```SQL
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

- **`created_at`:** Columna para registrar la fecha y hora de creación del registro.
- **`TIMESTAMP`:** Tipo de dato especial para fecha y hora.
- **`DEFAULT CURRENT_TIMESTAMP`:** Asigna de forma automática la fecha y la hora exactas del servidor en el momento preciso en que el contacto es guardado.

Línea 10

```SQL
);
```

- Cierra el paréntesis donde se definieron las columnas de la tabla y finaliza la instrucción SQL con punto y coma (`;`).

💡 Resumen clave para explicar en clase

Esta estructura demuestra tres conceptos fundamentales de bases de datos relacionales:

1. **Identificador Único (`PRIMARY KEY` + `AUTO_INCREMENT`):** Garantiza que no existan contactos duplicados a nivel de registro.
2. **Validación de Datos a Nivel de BD (`NOT NULL`, `ENUM`):** Mantiene la integridad de los datos impidiendo campos obligatorios vacíos o valores fuera de las categorías permitidas.
3. **Auditoría Automática (`DEFAULT CURRENT_TIMESTAMP`):** Despreocupa a la aplicación de enviar fechas manualmente desde PHP.

Database.php

Apertura y Declaración de la Clase

```PHP
<?php
```

- **Línea 1:** Etiqueta de apertura de PHP. Le indica al servidor que el código a continuación debe ser procesado por el intérprete de PHP.

```PHP
class Database {
```

- **Línea 2:** Declara la clase llamada `Database`. En POO, la clase es el "molde" o plantilla. Esta clase en particular tiene la única responsabilidad de gestionar la conexión con el servidor de MySQL.

Atributos (Propiedades) y Encapsulamiento

```PHP
   private $host = "localhost";
```

- **Línea 3:** Declara la propiedad `$host` con el valor `"localhost"`. Al tener el modificador de visibilidad `private`, aplicamos el pilar de **Encapsulamiento**: esta variable solo puede ser leída o modificada dentro de esta misma clase.

```PHP
   private $db_name = "agenda_db";
```

- **Línea 4:** Propiedad privada que guarda el nombre de la base de datos a la que se va a conectar la aplicación (`agenda_db`).

```PHP
   private $username = "root";
```

- **Línea 5:** Propiedad privada con el usuario de acceso a MySQL (en entornos de desarrollo local suele ser `root`).

```PHP
   private $password = "";
```

- **Línea 6:** Propiedad privada con la clave del usuario de MySQL (por defecto suele estar vacía `""` en servidores locales).

```
PHP
   private $conn;
```

- **Línea 7:** Propiedad privada destinada a almacenar el objeto de la conexión una vez que se establezca la comunicación con la base de datos.

Métodos (Comportamientos): `getConnection()`

```
PHP
   public function getConnection() {
```

- **Línea 9:** Define el método público `getConnection()`. Usamos `public` para que pueda ser invocado desde fuera de la clase (por ejemplo, desde `index.php` para obtener el enlace activo a la base de datos).

```
PHP
       $this->conn = null;
```

- **Línea 10:** Limpia la propiedad `$conn` asignándole `null` antes de intentar conectar.
- **`$this`:** Palabra clave que hace referencia a la instancia del objeto actual.

Conexión Segura con `try / catch` y PDO

```
PHP
       try {
```

- **Línea 12:** Inicia un bloque `try` ("intentar"). Contiene el código propenso a fallos (como establecer una conexión de red). Si ocurre un error, la ejecución no detiene el programa drásticamente, sino que salta al bloque `catch`.

```
PHP
           $this->conn = new PDO(
                "mysql:host=" . $this->host . ";dbname=" .
$this->db_name,
                $this->username,
```

```
                $this->password
            );
```

- **Líneas 13 a 17:** Instancia la clase nativa **PDO** (*PHP Data Objects*).
    - `new PDO(...)`: Crea un nuevo objeto de conexión.
    - Concatena las propiedades privadas para construir la cadena de conexión (DSN): `"mysql:host=localhost;dbname=agenda_db"`.
    - Envía el usuario y la contraseña como segundo y tercer parámetro.

```
PHP
            $this->conn->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);
```

- **Línea 18:** Le ordena a PDO que ante cualquier fallo SQL (error de sintaxis, tabla inexistente, etc.) **lance una excepción** (`PDO::ERRMODE_EXCEPTION`), facilitando la depuración del código en desarrollo.

```
PHP
        } catch(PDOException $exception) {
```

- **Línea 19:** Inicia el bloque `catch`. Si ocurrió un fallo al intentar conectar dentro del `try`, captura el objeto del error dentro de la variable `$exception`.

```
PHP
            echo "Error de conexión: " . $exception->getMessage();
```

- **Línea 20:** Imprime un mensaje claro en pantalla concatenando el detalle técnico del error recuperado con `$exception->getMessage()`.

```
PHP
        }
```

- **Línea 21:** Cierra el bloque `catch`.

Retorno de la Conexión y Cierre de la Clase

```
PHP
        return $this->conn;
```

- **Línea 23:** Retorna el enlace de conexión guardado en `$this->conn` para que pueda ser utilizado por los modelos del sistema (como `Contact.php`).

```
PHP
    }
}
```

- **Líneas 24 y 25:** Cierran el método `getConnection()` y la definición de la clase `Database`.

💡 Resumen para la clase

1. **Encapsulamiento:** Oculta datos sensibles de configuración (`$host`, `$username`, `$password`) usando `private`.
2. **Reutilización:** Permite conectar cualquier modelo a la base de datos `agenda_db` simplemente llamando a `getConnection()`.
3. **Manejo de Excepciones:** Atrapa los errores de conexión usando `try / catch` para evitar exponer información vulnerable en pantalla.

Contact.php

Apertura y Declaración de la Clase

```
PHP
<?php
```

- **Línea 1:** Abre la etiqueta de PHP para indicarle al servidor que procese el código a continuación.

```
PHP
class Contact {
```

- **Línea 2:** Declara la clase `Contact`. Esta clase representa el **Modelo** de un contacto dentro del sistema (su estructura y qué acciones puede realizar sobre la base de datos).

Propiedades (Atributos) y Encapsulamiento

```
PHP
    private $conn;
```

- **Línea 3:** Declaración de la propiedad `$conn` con visibilidad `private`. Guardará la conexión activa a MySQL (PDO). Es privada porque la conexión sólo debe ser manipulada por los métodos internos de esta clase.

```
PHP
   private $table_name = "contacts";
```

- **Línea 4:** Almacena el nombre exacto de la tabla en MySQL (`contacts`). Es `private` para evitar que se modifique desde fuera. Si el nombre de la tabla cambia en la base de datos, solo se actualiza en esta línea.

```
PHP
    public $id;
    public $name;
    public $phone;
    public $email;
    public $category;
```

- **Líneas 7 a 11:** Propiedades públicas que representan las columnas de la tabla:
  - `$id`: Identificador único del contacto.
  - `$name`: Nombre completo.
  - `$phone`: Número de teléfono.
  - `$email`: Correo electrónico.
  - `$category`: Categoría asignada (`Familia`, `Trabajo`, etc.).
- **Concepto para alumnos:** Son `public` para que desde el controlador o vista (`index.php`) se les puedan asignar valores directamente (ej: `$contact->name = $_POST['name'];`).

El Constructor

```
PHP
   public function __construct($db) {
        $this->conn = $db;
    }
```

- **Líneas 13 a 15:**
  - `__construct($db)`: Método especial que se ejecuta automáticamente cuando hacemos `new Contact($db)`.
  - `$this->conn = $db;`: Asigna la conexión a la base de datos que recibe como parámetro (`$db`) a la propiedad privada `$conn`.

- **Concepto para alumnos (Inyección de Dependencias):** La clase no genera una conexión nueva, sino que recibe una existente para reutilizar recursos.

1. Método de Lectura y Búsqueda: `read()`

```
PHP
   public function read($search = "") {
```

- **Línea 18:** Define el método `read()`. Acepta un parámetro opcional `$search` cuyo valor por defecto es una cadena vacía `""`.

```
PHP
       if (!empty($search)) {
```

- **Línea 19:** Evalúa si el usuario ingresó algún texto en la barra de búsqueda.

```
PHP
           $query = "SELECT * FROM " . $this->table_name . "
                     WHERE name LIKE :search OR phone LIKE :search OR
email LIKE :search
                     ORDER BY name ASC";
```

- **Líneas 21 a 23:** Define la consulta SQL de búsqueda. Usa el operador `LIKE` con un marcador de posición `:search` para filtrar coincidencias por nombre, teléfono o correo, ordenando el resultado alfabéticamente (`ORDER BY name ASC`).

```
PHP
           $stmt = $this->conn->prepare($query);
```

- **Línea 24:** Prepara la sentencia SQL dentro de PDO.

```
PHP
           $search_term = "%{$search}%";
            $stmt->bindParam(":search", $search_term);
```

- **Líneas 25 y 26:**
  - `$search_term = "%{$search}%";`: Agrega los comodines `%` al texto buscado. Esto permite encontrar coincidencias aunque la palabra esté al inicio, medio o final.

- `bindParam(":search", $search_term);`: Vincula de forma segura el término de búsqueda con el marcador `:search`.

```
PHP
       } else {
            $query = "SELECT * FROM " . $this->table_name . " ORDER BY
name ASC";
            $stmt = $this->conn->prepare($query);
        }
```

- **Líneas 27 a 30:** Si `$search` estaba vacío, prepara una consulta SQL simple para obtener **todos** los contactos ordenados alfabéticamente.

```
PHP
       $stmt->execute();
        return $stmt;
    }
```

- **Líneas 32 a 34:** Ejecuta la consulta SQL y retorna el objeto `$stmt` con los resultados listos para ser recorridos en la vista.

2. Método de Creación: `create()`

```
PHP
   public function create() {
```

- **Línea 37:** Método encargado de registrar un nuevo contacto.

```
PHP
       $query = "INSERT INTO " . $this->table_name . " (name, phone,
email, category)
                  VALUES (:name, :phone, :email, :category)";
        $stmt = $this->conn->prepare($query);
```

- **Líneas 38 a 40:** Define la consulta SQL `INSERT` usando marcadores de posición (`:name`, `:phone`, etc.) y la prepara en PDO.

```
PHP
       $this->name = htmlspecialchars(strip_tags($this->name));
        $this->phone = htmlspecialchars(strip_tags($this->phone));
```

```
        $this->email = htmlspecialchars(strip_tags($this->email));
        $this->category = htmlspecialchars(strip_tags($this->category));
```

- **Líneas 43 a 46 (Sanitización):**
    - `strip_tags()`: Elimina cualquier etiqueta HTML o PHP inyectada por el usuario.
    - `htmlspecialchars()`: Convierte caracteres especiales en entidades HTML.
    - **Concepto para alumnos:** Protege la aplicación contra ataques **XSS (Cross-Site Scripting)**.

```
PHP
        $stmt->bindParam(":name", $this->name);
        $stmt->bindParam(":phone", $this->phone);
        $stmt->bindParam(":email", $this->email);
        $stmt->bindParam(":category", $this->category);
```

- **Líneas 49 a 52:** Vincula cada propiedad limpiecita del objeto con su correspondiente marcador de posición en la consulta SQL. Evita **SQL Injection**.

```
PHP
        return $stmt->execute();
    }
```

- **Línea 54:** Ejecuta la inserción en MySQL y retorna `true` si fue exitosa o `false` si falló.

3. Método de Eliminación: `delete()`

```
PHP
    public function delete() {
```

- **Línea 57:** Método para eliminar un contacto por su ID.

```
PHP
        $query = "DELETE FROM " . $this->table_name . " WHERE id = :id";
        $stmt = $this->conn->prepare($query);
```

- **Líneas 58 y 59:** Define y prepara la consulta `DELETE FROM contacts WHERE id = :id`.

```
PHP
        $stmt->bindParam(":id", $this->id);
```

- **Línea 60:** Vincula el ID asignado a la propiedad `$this->id` con el marcador `:id`.

```
PHP
        return $stmt->execute();
    }
}
```

- **Líneas 61 y 62:** Ejecuta el borrado en MySQL, retorna el resultado booleano y cierra la definición de la clase `Contact`.

💡 Resumen pedagógico para destacar en clase

1. **Reutilización del método `read()`:** Muestra cómo un solo método puede resolver dos necesidades (listar todo o buscar) usando un parámetro con valor por defecto (`$search = ""`).
2. **Seguridad en dos capas:**
    - **Sanitización** con `strip_tags()` + `htmlspecialchars()` en PHP (previene XSS).
    - **Consultas Preparadas** con `prepare()` + `bindParam()` en PDO (previene SQL Injection).
3. **Mantenibilidad:** Toda la lógica referente a la tabla `contacts` está centralizada en esta clase.

index.php

1. Carga de Dependencias e Inicialización de Objetos

PHP

```
<?php
require_once "Database.php";
require_once "Contact.php";
```

- **Líneas 2 y 3:** Importan las clases `Database` (conexión a la base de datos) y `Contact` (modelo del contacto). Se usa `require_once` para garantizar que no se duplique la carga de archivos.

```
PHP
```

```php
$database = new Database();
$db = $database->getConnection();
$contact = new Contact($db);
```

- **Línea 5:** Crea una nueva instancia (objeto) de la clase `Database`.
- **Línea 6:** Llama al método `getConnection()` para establecer el enlace con MySQL y guardarlo en la variable `$db`.
- **Línea 7:** Instancia la clase `Contact` pasándole la conexión activa `$db` (Inyección de Dependencias).

```php
PHP
$message = "";
$messageType = "";
```

- **Líneas 10 y 11:** Inicializan dos variables que servirán para mostrar mensajes de retroalimentación visual (éxito o error) al usuario en la interfaz.

2. Procesamiento del Formulario (Creación - POST)

```php
PHP
if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST["action"]) &&
$_POST["action"] == "add") {
```

- **Línea 14:** Verifica tres cosas: que el formulario se envió por el método **POST**, que existe la variable `action` y que su valor es `"add"`.

```php
PHP
   if (!empty($_POST["name"]) && !empty($_POST["phone"])) {
        $contact->name = $_POST["name"];
        $contact->phone = $_POST["phone"];
        $contact->email = $_POST["email"] ?? "";
        $contact->category = $_POST["category"] ?? "Otros";
```

- **Línea 15:** Comprueba que los campos obligatorios (`name` y `phone`) no lleguen vacíos.
- **Líneas 16 a 19:** Asigna los valores recibidos del formulario a las propiedades del objeto `$contact`. Se utiliza el **operador de fusión de nulos (`??`)** para asignar un valor por defecto si los campos opcionales no son enviados.

```php
PHP
```

```
        if ($contact->create()) {
            $message = "¡Contacto guardado exitosamente!";
            $messageType = "success";
        } else {
            $message = "Ocurrió un error al guardar el contacto.";
            $messageType = "error";
        }
    }
}
```

- **Líneas 21 a 27:** Ejecuta el método `create()` de la clase `Contact`. Si retorna `true`, configura el mensaje de éxito; si falla, asigna el mensaje de error.

3. Procesamiento de Eliminación (GET) y Búsqueda

```
PHP
if (isset($_GET["action"]) && $_GET["action"] == "delete" &&
isset($_GET["id"])) {
    $contact->id = $_GET["id"];
    $contact->delete();
    header("Location: index.php");
    exit();
}
```

- **Línea 32:** Verifica si se hizo clic en un enlace de eliminación que envía por la URL (`GET`) los parámetros `action=delete` e `id`.
- **Líneas 33 y 34:** Asigna el ID al objeto y llama a `$contact->delete()` para borrarlo de la base de datos.
- **Líneas 35 y 36:** Redirige de vuelta a `index.php` con `header()` y detiene la ejecución con `exit()`. Esto evita que al recargar la página el navegador intente eliminar el mismo registro.

```
PHP
$search_query = $_GET["search"] ?? "";
$stmt = $contact->read($search_query);
?>
```

- **Línea 40:** Captura el texto enviado en el buscador mediante `GET`. Si no hay búsqueda, asigna una cadena vacía `""`.
- **Línea 41:** Ejecuta el método `read()` del modelo pasándole la consulta de búsqueda. Guarda el resultado de PDO en la variable `$stmt`.

4. Estructura HTML y Encabezado de la Aplicación

```HTML
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Agenda de Contactos POO</title>
  <link rel="stylesheet" href="styles.css">
```

- **Líneas 44 a 49:** Configuración básica del documento HTML5 y vinculación de la hoja de estilos externa `styles.css`.

```HTML
 <header class="navbar">
    <div class="logo">
      <span class="logo-icon">📇</span> ContactApp <span
class="badge-mini">POO</span>
    </div>
    <div class="nav-info">
      <span class="status-dot"></span> MySQL Conectado
    </div>
  </header>
```

- **Líneas 57 a 64:** Barra de navegación superior estática con el logotipo y la indicación visual de conexión a la base de datos.

5. Mensajes de Feedback e Interfaz Principal

```PHP
<?php if(!empty($message)): ?>
  <div class="alert alert-<?= $messageType ?>">
    <?= htmlspecialchars($message) ?>
  </div>
<?php endif; ?>
```

- **Líneas 74 a 78:** Evalúa si existe un mensaje en `$message`. De ser así, imprime una caja de alerta dinámica con la clase CSS correspondiente (`alert-success` o `alert-error`).

```HTML
<form action="index.php" method="POST" class="contact-form">
```

```
  <input type="hidden" name="action" value="add">
```

- **Líneas 89 y 90:** Formulario para agregar registros. El campo oculto `type="hidden"` envía `name="action"` con el valor `"add"` para que la lógica PHP sepa qué acción ejecutar.

HTML

<label for="name">Nombre Completo *</label>
<input type="text" id="name" name="name" required placeholder="Ej. Ana Gómez" autocomplete="off">

- **Líneas 92 a 95:** Campo de entrada de texto obligatorio (`required`) para el nombre. Los demás campos (`phone`, `email`, `category`) siguen esta misma estructura.

6. Buscador y Recorrido de Contactos (Lista)

```
HTML
<form action="index.php" method="GET" class="search-bar">
  <input type="text" name="search" placeholder="Buscar..." value="<?= htmlspecialchars($search_query) ?>">
  <button type="submit" class="btn btn-secondary">Buscar</button>
  <?php if(!empty($search_query)): ?>
    <a href="index.php" class="btn btn-icon" title="Limpiar búsqueda">✕</a>
  <?php endif; ?>
</form>
```

- **Líneas 124 a 130:** Formulario de búsqueda rápida que envía la consulta vía **GET**. Si hay una búsqueda activa, muestra un botón para limpiar el filtro y recargar la lista completa.

```
PHP
<?php
$hasContacts = false;
while ($row = $stmt->fetch(PDO::FETCH_ASSOC)):
  $hasContacts = true;
?>
```

- **Línea 135:** Inicializa el indicador `$hasContacts` en `false`.

- **Línea 136:** Bucle `while` que extrae fila por fila los registros traídos por PDO (`$stmt->fetch(PDO::FETCH_ASSOC)`) en forma de arreglo asociativo.
- **Línea 137:** Cambia `$hasContacts` a `true` al confirmar que hay al menos un registro.

HTML
```
<div class="contact-avatar">
  <?= strtoupper(substr($row['name'], 0, 1)) ?>
</div>
```

- **Línea 141:** Extrae la primera letra del nombre con `substr()` y la convierte a mayúscula con `strtoupper()` para generar un avatar con iniciales de forma dinámica.

HTML
```
<h3><?= htmlspecialchars($row['name']) ?></h3>
<span class="badge-cat badge-<?= strtolower($row['category']) ?>">
  <?= htmlspecialchars($row['category']) ?>
</span>
```

- **Líneas 145 a 148:** Imprime el nombre y la categoría aplicando `htmlspecialchars()` para proteger la vista contra ataques de script (XSS).

HTML
```
<a href="index.php?action=delete&id=<?= $row['id'] ?>"
   class="btn-delete"
   onclick="return confirm('¿Seguro que deseas eliminar este contacto?')">
  🗑️
</a>
```

- **Líneas 157 a 161:** Botón/enlace para eliminar. Construye una URL dinámica con el ID de la fila (`index.php?action=delete&id=X`) y dispara un cuadro de confirmación nativo en JavaScript (`confirm()`) antes de ejecutar la acción.

PHP
```
 <?php endwhile; ?>

  <?php if (!$hasContacts): ?>
    <div class="empty-state">
      <span class="empty-icon">🔍</span>
      <p>No se encontraron contactos en la agenda.</p>
    </div>
  <?php endif; ?>
```

- **Línea 163:** Cierra el bucle `while`.
- **Líneas 165 a 170:** Si `$hasContacts` permaneció en `false`, muestra una tarjeta informativa indicando que la lista está vacía o que la búsqueda no arrojó coincidencias.

💡 Resumen de arquitectura para tus alumnos

- **Separación de Responsabilidades:** `index.php` actúa como el **Controlador** (procesa la entrada del usuario) y como la **Vista** (dibuja la pantalla en HTML), mientras que `Contact.php` y `Database.php` son el **Modelo** y la **Conexión**.
- **Seguridad:** Uso del patrón **Post/Redirect/Get** tras eliminar registros para evitar reenvíos accidentales, y sanitización de datos impresos con `htmlspecialchars()`.

styles.css

1. Variables de Diseño (Design Tokens) — Seudoclase `:root`

CSS

```
:root {
  --font-family: 'Plus Jakarta Sans', -apple-system, BlinkMacSystemFont, sans-serif;
  ...
}
```

- **`:root`:** Representa el nivel más alto del árbol HTML (`<html>`). Define variables globales que estarán disponibles en todo el documento.
- **`--font-family`:** Tipografía principal con fuentes del sistema de respaldo (*fallbacks*) en caso de no cargar Google Fonts.
- **`--bg-main`, `--bg-card`, `--bg-card-hover`:** Paleta de colores para el fondo de la página e interfaces flotantes. Usan `rgba()` para añadir transparencia (canal alfa).
- **`--border-color` y `--border-color-hover`:** Colores de los bordes sutiles en estado normal y al pasar el cursor sobre las tarjetas.
- **`--primary`, `--primary-hover`, `--primary-glow`:** El color de acento (azul/violeta), su versión oscura para interacciones (*hover*) y un color semitransparente para simular un resplandor luminoso (*glow*).
- **`--text-main` y `--text-muted`:** Blanco suave para textos principales y gris claro para textos secundarios.
- **`--success` y `--danger`:** Colores semánticos verde (éxito/confirmación) y rojo (errores/alertas/eliminación).
- **`--radius-sm`, `--radius-md`, `--radius-lg`:** Radios de curvatura estandarizados para esquinas de botones, tarjetas y campos de texto.
- **`--shadow-main`:** Sombra proyectada profunda con desplazamiento vertical y difuminado.

- **`--transition`:** Define una animación suave de `0.3s` con la curva de velocidad `cubic-bezier(0.4, 0, 0.2, 1)` para todas las transiciones de la aplicación.

2. Reseteo Universal y Estilos Base

CSS
```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

- **`*` (Selector universal):** Elimina los márgenes y rellenos que los navegadores aplican por defecto a todas las etiquetas HTML.
- **`box-sizing: border-box;`:** Ajusta el modelo de caja. Garantiza que el ancho y alto declarados en un elemento incluyan el *padding* y el *border* dentro de las medidas totales.

CSS
```
body {
  font-family: var(--font-family);
  background-color: var(--bg-main);
  color: var(--text-main);
  min-height: 100vh;
  line-height: 1.6;
  overflow-x: hidden;
  background-image:
    radial-gradient(at 0% 0%, rgba(99, 102, 241, 0.15) 0px, transparent 50%),
    radial-gradient(at 100% 100%, rgba(168, 85, 247, 0.15) 0px, transparent 50%);
}
```

- **`font-family` / `background-color` / `color`:** Hereda la tipografía, el fondo oscuro y el color de texto desde las variables de `:root`.
- **`min-height: 100vh;`:** Asegura que el cuerpo ocupe como mínimo el 100% del alto de la pantalla visible (*viewport height*).
- **`line-height: 1.6;`:** Altura de línea para mejorar la legibilidad del texto.
- **`overflow-x: hidden;`:** Oculta cualquier desbordamiento horizontal para evitar la barra de desplazamiento lateral en dispositivos móviles.
- **`background-image: radial-gradient(...)`:** Crea dos orbes de luz ambiental tenue en la esquina superior izquierda y posterior derecha mezclando colores morados y azules semitransparentes.

CSS
```
.container {
  max-width: 1100px;
```

```
  margin: 0 auto;
  padding: 0 24px 60px;
}
```

- **`max-width: 1100px;`:** Establece un ancho máximo seguro para que el contenido no se estire demasiado en monitores anchos.
- **`margin: 0 auto;`:** Centra horizontalmente la caja contenedora dentro de la ventana.
- **`padding: 0 24px 60px;`:** Añade espacio lateral (`24px`) para evitar que el contenido toque los bordes del navegador en pantallas pequeñas, y espacio inferior (`60px`).

3. Barra de Navegación (`.navbar`)

CSS
```
.navbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 16px 40px;
  backdrop-filter: blur(12px);
  background: rgba(11, 15, 25, 0.8);
  border-bottom: 1px solid var(--border-color);
  position: sticky;
  top: 0;
  z-index: 100;
}
```

- **`display: flex;` / `align-items: center;` / `justify-content: space-between;`:** Distribuye el logo a la izquierda y el indicador de conexión a la derecha, manteniéndolos alineados verticalmente al centro.
- **`backdrop-filter: blur(12px);`:** Aplica un efecto de desenfoque esmerilado (*Glassmorphism*) a todo el contenido que pase por detrás de la barra.
- **`background: rgba(11, 15, 25, 0.8);`:** Fondo semitransparente con un 80% de opacidad para acompañar el desenfoque.
- **`position: sticky;` / `top: 0;` / `z-index: 100;`:** Fija la barra en la parte superior al hacer *scroll*, asegurando que permanezca por encima del resto del contenido.

CSS
```
.logo { font-size: 1.25rem; font-weight: 700; display: flex; align-items: center; gap: 8px; }
.badge-mini { font-size: 0.7rem; background: var(--primary); color: #fff; padding: 2px 6px; border-radius: 4px; }
.nav-info { font-size: 0.85rem; color: var(--text-muted); display: flex; align-items: center; gap: 8px; }
```

- **`gap: 8px;`:** Separa homogéneamente los elementos secundarios dentro de un contenedor Flexbox sin usar márgenes manuales.
- **`.badge-mini`:** Pequeña etiqueta distintiva (ej. "POO") con fondo azul y bordes redondeados.

CSS
```
.status-dot {
  width: 8px;
  height: 8px;
  background-color: var(--success);
  border-radius: 50%;
  display: inline-block;
}
```

- **`border-radius: 50%;`:** Transforma un cuadro de `8px x 8px` en una luz/indicador circular perfecto de color verde para simular la conexión a la base de datos.

4. Sección Hero y Alertas de Estado

CSS
```
.hero { text-align: center; padding: 40px 20px 30px; max-width: 700px; margin: 0 auto; }
.badge { display: inline-block; padding: 6px 16px; border-radius: 50px; background: rgba(99, 102, 241, 0.1); border: 1px solid rgba(99, 102, 241, 0.3); color: #818cf8; font-size: 0.85rem; font-weight: 600; margin-bottom: 16px; }
.hero h1 { font-size: 2.5rem; font-weight: 700; line-height: 1.2; margin-bottom: 12px; }
```

- **`.hero`:** Encabezado con texto centrado y ancho restringido a `700px`.
- **`.badge`:** Etiqueta estilizada tipo píldora (`border-radius: 50px`) que resalta el carácter didáctico del proyecto.

CSS
```
.gradient-text {
  background: linear-gradient(135deg, #818cf8 0%, #c084fc 100%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
}
```

- **`background: linear-gradient(...)`:** Crea un degradado de color que va de azul violeta a morado claro en un ángulo de 135 grados.
- **`-webkit-background-clip: text;` / `-webkit-text-fill-color: transparent;`:** Recorta el fondo con la forma exacta de las letras y hace el relleno de texto transparente, logrando un título con texto degradado.

CSS
.alert { padding: 12px 20px; border-radius: var(--radius-sm); margin-bottom: 24px; text-align: center; font-weight: 500; }
.alert-success { background: rgba(16, 185, 129, 0.15); border: 1px solid var(--success); color: #6ee7b7; }
.alert-error { background: rgba(239, 68, 68, 0.15); border: 1px solid var(--danger); color: #fca5a5; }

- **`.alert`:** Modismos visuales de notificación. Generan cajas de alerta con color de fondo atenuado, bordes de color brillante y texto contrastado dependiendo del estado devuelto por PHP (`success` o `error`).

5. Maquetación Principal (`.app-grid` y `.card`)

CSS
.app-grid {
  display: grid;
  grid-template-columns: 1fr 1.5fr;
  gap: 24px;
  align-items: start;
}

- **`display: grid;`:** Habilita el sistema de cuadrícula en CSS.
- **`grid-template-columns: 1fr 1.5fr;`:** Divide la pantalla en dos columnas asimétricas usando fracciones (`fr`). La columna izquierda (formulario) toma una fracción y la derecha (lista) toma una fracción y media (proporción 40% / 60%).
- **`align-items: start;`:** Evita que ambas tarjetas se estiren a la misma altura si una tiene más contenido que la otra.

CSS
.card {
  background: var(--bg-card);
  backdrop-filter: blur(16px);
  border: 1px solid var(--border-color);
  border-radius: var(--radius-md);
  padding: 28px;
  box-shadow: var(--shadow-main);
}

- **`.card`:** Diseña el contenedor de las dos secciones del sistema mediante *Glassmorphism*: fondo con transparencia, desenfoque posterior de `16px`, borde fino de cristal y sombra profunda.

6. Formularios e Entradas de Datos

CSS
```
.contact-form { display: flex; flex-direction: column; gap: 16px; }
.form-group { display: flex; flex-direction: column; gap: 6px; }
.form-group label { font-size: 0.85rem; font-weight: 600; color: var(--text-muted); }
```

- **`flex-direction: column;`:** Apila verticalmente las etiquetas `<label>` e `<input>` dentro de cada grupo, y a su vez apila todos los grupos en el formulario.

CSS
```
input, select {
  font-family: inherit;
  background: rgba(11, 15, 25, 0.6);
  border: 1px solid var(--border-color);
  color: var(--text-main);
  padding: 10px 14px;
  border-radius: var(--radius-sm);
  font-size: 0.9rem;
  outline: none;
  transition: var(--transition);
}
```

- Estiliza todos los campos de texto e interactivos con fondo semitransparente oscuro, tipografía idéntica a la aplicación (`font-family: inherit`) y elimina el borde azul brillante genérico del navegador (`outline: none`).

CSS
```
input:focus, select:focus {
  border-color: var(--primary);
  box-shadow: 0 0 0 3px rgba(99, 102, 241, 0.2);
}
```

- **`:focus`:** Cuando el usuario hace clic o se posiciona sobre un campo, cambia el borde al azul de la aplicación y crea un halo de luz alrededor para dar retroalimentación de enfoque.

7. Botones e Interacciones

CSS
```
.btn {
  font-family: inherit;
  font-weight: 600;
  border-radius: var(--radius-sm);
  padding: 10px 18px;
  border: none;
  cursor: pointer;
```

```
  transition: var(--transition);
  display: inline-flex;
  align-items: center;
  justify-content: center;
  text-decoration: none;
}
```

- Base reutilizable para cualquier botón o enlace con apariencia de botón en la aplicación.

CSS
```
.btn-primary { background-color: var(--primary); color: #ffffff; box-shadow: 0 4px 14px var(--primary-glow); }
.btn-primary:hover { background-color: var(--primary-hover); }
.btn-secondary { background: rgba(255, 255, 255, 0.08); color: var(--text-main); border: 1px solid var(--border-color); }
.btn-secondary:hover { background: rgba(255, 255, 255, 0.15); }
.btn-block { width: 100%; margin-top: 8px; }
```

- Variaciones de jerarquía: `.btn-primary` para la acción principal (guardar) y `.btn-secondary` para acciones de apoyo (buscar). `.btn-block` expande el botón al 100% del ancho del formulario.

8. Lista de Contactos, Avatares y Estado Vacío

CSS
```
.search-bar { display: flex; gap: 8px; margin-bottom: 20px; }
.search-bar input { flex: 1; }
```

- **`flex: 1;`:** Le ordena al campo de texto del buscador que tome todo el espacio horizontal sobrante disponible, empujando el botón "Buscar" a la derecha.

CSS
```
.contact-item {
  background: rgba(11, 15, 25, 0.4);
  border: 1px solid var(--border-color);
  border-radius: var(--radius-sm);
  padding: 14px 16px;
  display: flex;
  align-items: center;
  gap: 14px;
  transition: var(--transition);
}

.contact-item:hover {
  border-color: var(--border-color-hover);
```

```
  background: var(--bg-card-hover);
}
```

- **`.contact-item`:** Fila individual de cada contacto guardado. Al pasar el puntero (`:hover`), se aclara ligeramente el fondo y resalta el borde con un tono brillante.

CSS
```
.contact-avatar {
  width: 40px;
  height: 40px;
  border-radius: 50%;
  background: linear-gradient(135deg, var(--primary), #a855f7);
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: 700;
  font-size: 1.1rem;
  color: white;
  flex-shrink: 0;
}
```

- **`.contact-avatar`:** Crea la burbuja redonda (`border-radius: 50%`) que muestra la inicial del contacto sobre un fondo en degradado violeta. `flex-shrink: 0` impide que el círculo se aplaste si el texto del nombre es muy largo.

CSS
```
.badge-cat { font-size: 0.7rem; padding: 2px 8px; border-radius: 12px; font-weight: 600; background: rgba(255, 255, 255, 0.1); color: var(--text-muted); }
.btn-delete { text-decoration: none; font-size: 1rem; padding: 6px 10px; border-radius: 6px; transition: var(--transition); opacity: 0.7; }
.btn-delete:hover { opacity: 1; background: rgba(239, 68, 68, 0.2); }
```

- **`.badge-cat`:** Píldora que indica la categoría del contacto (`Familia`, `Trabajo`, etc.).
- **`.btn-delete`:** Botón discreto para eliminar que resalta con un fondo rojo transparente cuando el usuario pasa el mouse por encima.

9. Adaptabilidad Móvil (Responsive Design)

CSS
```
@media (max-width: 850px) {
  .app-grid {
    grid-template-columns: 1fr;
  }
}
```

- `@media (max-width: 850px)` **(Media Query):** Detecta cuando la pantalla del usuario mide `850px` o menos (tabletas o celulares).
- `grid-template-columns: 1fr;`**:** Cambia la cuadrícula de 2 columnas a **1 sola columna**. El formulario se colocará arriba y la lista de contactos justo abajo, adaptando la vista perfectamente a dispositivos móviles.

---

1. Catálogo / Inventario de Productos (Nivel: Principiante - Medio)

Un clásico del desarrollo web que amplía la lógica del *To-Do List*.

- **¿De qué trata?:** Sistema para registrar, editar y listar productos (por ejemplo, una tienda de tecnología o libros) con nombre, precio, stock y categoría.
- **Nuevos conceptos de POO / PHP a enseñar:**
  - **Manejo de archivos (Uploads):** Guardar imágenes de productos en el servidor y su ruta en la base de datos.
  - **Métodos con filtros:** Crear métodos en la clase `Product` como `readByCategory($category_id)` o `searchByTitle($keyword)`.
  - **Relación de Tablas en MySQL:** Unir la tabla `products` con `categories` mediante una clave foránea (`FOREIGN KEY`).

📁 Estructura del Proyecto

Plaintext
```
inventario_poo/
├── schema.sql        # Script de la base de datos (con Foreign Keys)
├── Database.php      # Clase de conexión PDO
├── Category.php      # Modelo de Categorías
├── Product.php       # Modelo de Productos (con Upload de imágenes)
├── index.php         # Controlador + Vista (Interfaz moderna)
├── styles.css        # Estilos CSS (Dark Mode / Glassmorphism)
└── uploads/          # Carpeta para almacenar las imágenes subidas
```

1. Base de Datos con Claves Foráneas (`schema.sql`)

En este script creamos la relación de **Uno a Muchos (1:N)**: una categoría puede tener muchos productos.

SQL
```
CREATE DATABASE IF NOT EXISTS inventario_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

```
USE inventario_db;

-- Tabla 1: Categorías
CREATE TABLE IF NOT EXISTS categories (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tabla 2: Productos con Clave Foránea (FOREIGN KEY)
CREATE TABLE IF NOT EXISTS products (
    id INT AUTO_INCREMENT PRIMARY KEY,
    category_id INT NOT NULL,
    name VARCHAR(150) NOT NULL,
    description TEXT NULL,
    price DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL DEFAULT 0,
    image VARCHAR(255) NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_category FOREIGN KEY (category_id) REFERENCES categories(id)
ON DELETE CASCADE
);

-- Datos iniciales de categoría
INSERT INTO categories (name) VALUES
('Tecnología'),
('Libros'),
('Ropa y Calzado'),
('Hogar y Oficina');
```

2. Conexión a la Base de Datos (`Database.php`)

PHP

```
<?php
class Database {
    private $host = "localhost";
    private $db_name = "inventario_db";
    private $username = "root";
    private $password = "";
    private $conn;

    public function getConnection() {
        $this->conn = null;

        try {
            $this->conn = new PDO(
                "mysql:host=" . $this->host . ";dbname=" . $this->db_name,
```

```
            $this->username,
            $this->password
        );
        $this->conn->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);
    } catch(PDOException $exception) {
        echo "Error de conexión: " . $exception->getMessage();
    }

    return $this->conn;
  }
}
```

3. Modelo Categoría (`Category.php`)

```
PHP
<?php
class Category {
  private $conn;
  private $table_name = "categories";

  public $id;
  public $name;

  public function __construct($db) {
    $this->conn = $db;
  }

  // Obtener todas las categorías para llenar los selectores en la vista
  public function readAll() {
    $query = "SELECT * FROM " . $this->table_name . " ORDER BY name ASC";
    $stmt = $this->conn->prepare($query);
    $stmt->execute();
    return $stmt;
  }
}
```

4. Modelo Producto (`Product.php`)

Este modelo incorpora la relación `JOIN` en SQL, filtros dinámicos y la lógica de **procesamiento y subida de imágenes**:

```
PHP
<?php
class Product {
  private $conn;
```

```
private $table_name = "products";

public $id;
public $category_id;
public $name;
public $description;
public $price;
public $stock;
public $image;

public function __construct($db) {
    $this->conn = $db;
}

// 1. LEER PRODUCTOS CON FILTROS Y JOIN
public function read($category_id = null, $keyword = null) {
    $query = "SELECT p.*, c.name as category_name
            FROM " . $this->table_name . " p
            LEFT JOIN categories c ON p.category_id = c.id
            WHERE 1=1";

    if (!empty($category_id)) {
        $query .= " AND p.category_id = :category_id";
    }

    if (!empty($keyword)) {
        $query .= " AND (p.name LIKE :keyword OR p.description LIKE :keyword)";
    }

    $query .= " ORDER BY p.id DESC";

    $stmt = $this->conn->prepare($query);

    if (!empty($category_id)) {
        $stmt->bindParam(":category_id", $category_id);
    }

    if (!empty($keyword)) {
        $keyword_param = "%{$keyword}%";
        $stmt->bindParam(":keyword", $keyword_param);
    }

    $stmt->execute();
    return $stmt;
}

// 2. CREAR PRODUCTO
public function create() {
```

```
    $query = "INSERT INTO " . $this->table_name . "
            (category_id, name, description, price, stock, image)
            VALUES (:category_id, :name, :description, :price, :stock, :image)";

    $stmt = $this->conn->prepare($query);

    // Sanitización
    $this->category_id = htmlspecialchars(strip_tags($this->category_id));
    $this->name = htmlspecialchars(strip_tags($this->name));
    $this->description = htmlspecialchars(strip_tags($this->description));
    $this->price = htmlspecialchars(strip_tags($this->price));
    $this->stock = htmlspecialchars(strip_tags($this->stock));
    $this->image = htmlspecialchars(strip_tags($this->image));

    // Vinculación
    $stmt->bindParam(":category_id", $this->category_id);
    $stmt->bindParam(":name", $this->name);
    $stmt->bindParam(":description", $this->description);
    $stmt->bindParam(":price", $this->price);
    $stmt->bindParam(":stock", $this->stock);
    $stmt->bindParam(":image", $this->image);

    return $stmt->execute();
}

// 3. MÉTODO DE SUBIDA DE IMAGEN
public function uploadImage($file) {
    if (isset($file) && $file['error'] === UPLOAD_ERR_OK) {
        $upload_dir = "uploads/";

        if (!is_dir($upload_dir)) {
            mkdir($upload_dir, 0777, true);
        }

        $file_extension = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));
        $allowed_extensions = ['jpg', 'jpeg', 'png', 'webp', 'gif'];

        if (in_array($file_extension, $allowed_extensions)) {
            // Nombre único para evitar sobrescribir archivos
            $new_filename = uniqid('prod_') . '.' . $file_extension;
            $target_file = $upload_dir . $new_filename;

            if (move_uploaded_file($file['tmp_name'], $target_file)) {
                return $new_filename;
            }
        }
    }
    return null;
```

```
    }

    // 4. ELIMINAR PRODUCTO (Y SU IMAGEN ASOCIADA)
    public function delete() {
        // Obtener ruta de la imagen antes de borrar
        $query = "SELECT image FROM " . $this->table_name . " WHERE id = :id";
        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(":id", $this->id);
        $stmt->execute();
        $row = $stmt->fetch(PDO::FETCH_ASSOC);

        if ($row && !empty($row['image'])) {
            $image_path = "uploads/" . $row['image'];
            if (file_exists($image_path)) {
                unlink($image_path); // Elimina el archivo físico del servidor
            }
        }

        // Eliminar registro en BD
        $query = "DELETE FROM " . $this->table_name . " WHERE id = :id";
        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(":id", $this->id);
        return $stmt->execute();
    }
}
```

5. Controlador y Vista (`index.php`)

PHP

```
<?php
require_once "Database.php";
require_once "Category.php";
require_once "Product.php";

$database = new Database();
$db = $database->getConnection();

$categoryModel = new Category($db);
$productModel = new Product($db);

$message = "";
$messageType = "";

// 1. Procesar Registro de Producto
if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST["action"]) &&
$_POST["action"] == "add") {
```

```
    if (!empty($_POST["name"]) && !empty($_POST["price"]) &&
!empty($_POST["category_id"])) {

        $productModel->name = $_POST["name"];
        $productModel->category_id = $_POST["category_id"];
        $productModel->price = $_POST["price"];
        $productModel->stock = $_POST["stock"] ?? 0;
        $productModel->description = $_POST["description"] ?? "";

        // Manejo de la imagen
        $image_name = $productModel->uploadImage($_FILES["image"] ?? null);
        $productModel->image = $image_name ?? "";

        if ($productModel->create()) {
            $message = "¡Producto registrado en el inventario con éxito!";
            $messageType = "success";
        } else {
            $message = "Error al guardar el producto.";
            $messageType = "error";
        }
    }
}

// 2. Procesar Eliminación de Producto
if (isset($_GET["action"]) && $_GET["action"] == "delete" && isset($_GET["id"])) {
    $productModel->id = $_GET["id"];
    $productModel->delete();
    header("Location: index.php");
    exit();
}

// 3. Capturar Filtros
$selected_category = $_GET["category_filter"] ?? null;
$search_keyword = $_GET["search"] ?? null;

$categories_stmt = $categoryModel->readAll();
$products_stmt = $productModel->read($selected_category, $search_keyword);
?>

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Catálogo e Inventario de Productos</title>
  <link rel="stylesheet" href="styles.css">
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
```

```
  <link
href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700
&display=swap" rel="stylesheet">
</head>
<body>

  <!-- Navegación -->
  <header class="navbar">
    <div class="logo">
      <span class="logo-icon">📦</span> InventarioPro <span
class="badge-mini">POO</span>
    </div>
    <div class="nav-info">
      <span class="status-dot"></span> Base de Datos Activa
    </div>
  </header>

  <main class="container">
    <section class="hero">
      <span class="badge">✨ Proyecto Nivel Intermedio</span>
      <h1>Catálogo de <span class="gradient-text">Productos</span></h1>
      <p class="hero-subtitle">Gestión de inventario con subida de imágenes, categorías con
Foreign Key y filtros dinámicos en PHP.</p>
    </section>

    <?php if(!empty($message)): ?>
      <div class="alert alert-<?= $messageType ?>">
        <?= htmlspecialchars($message) ?>
      </div>
    <?php endif; ?>

    <div class="app-grid">

      <!-- Formulario de Registro -->
      <div class="card card-form">
        <div class="card-header">
          <h2>➕ Registrar Producto</h2>
          <p>Ingresa los datos y sube una foto del producto</p>
        </div>

        <form action="index.php" method="POST" enctype="multipart/form-data"
class="contact-form">
          <input type="hidden" name="action" value="add">

          <div class="form-group">
            <label for="name">Nombre del Producto *</label>
            <input type="text" id="name" name="name" required placeholder="Ej. Laptop
Gaming 16GB RAM">
```

```
        </div>

        <div class="form-group">
          <label for="category_id">Categoría *</label>
          <select id="category_id" name="category_id" required>
            <option value="">-- Selecciona una categoría --</option>
            <?php
            // Reutilizamos la consulta para los desplegables
            $categories_stmt_form = $categoryModel->readAll();
            while($cat = $categories_stmt_form->fetch(PDO::FETCH_ASSOC)):
            ?>
              <option value="<?= $cat['id'] ?>"><?= htmlspecialchars($cat['name']) ?></option>
            <?php endwhile; ?>
          </select>
        </div>

        <div class="form-row">
          <div class="form-group">
            <label for="price">Precio ($) *</label>
            <input type="number" step="0.01" id="price" name="price" required
placeholder="299.99">
          </div>

          <div class="form-group">
            <label for="stock">Stock inicial *</label>
            <input type="number" id="stock" name="stock" required value="10">
          </div>
        </div>

        <div class="form-group">
          <label for="image">Imagen del Producto</label>
          <input type="file" id="image" name="image" accept="image/*">
        </div>

        <div class="form-group">
          <label for="description">Descripción</label>
          <textarea id="description" name="description" rows="3" placeholder="Breve detalle
de las características..."></textarea>
        </div>

        <button type="submit" class="btn btn-primary btn-block">Guardar Producto</button>
      </form>
    </div>

    <!-- Listado de Productos / Filtros -->
    <div class="card card-list">
      <div class="card-header">
        <h2>🛍️ Inventario de Productos</h2>
```

```
        <p>Filtra por categoría o busca por palabras clave</p>
      </div>

      <!-- Filtros Dinámicos -->
      <form action="index.php" method="GET" class="filter-bar">
        <input type="text" name="search" placeholder="Buscar producto..." value="<?=
htmlspecialchars($search_keyword ?? '') ?>">

        <select name="category_filter">
          <option value="">Todas las Categorías</option>
          <?php while($cat = $categories_stmt->fetch(PDO::FETCH_ASSOC)): ?>
            <option value="<?= $cat['id'] ?>" <?= ($selected_category == $cat['id']) ? 'selected' :
'' ?>>
              <?= htmlspecialchars($cat['name']) ?>
            </option>
          <?php endwhile; ?>
        </select>

        <button type="submit" class="btn btn-secondary">Filtrar</button>
        <?php if(!empty($search_keyword) || !empty($selected_category)): ?>
          <a href="index.php" class="btn btn-icon" title="Limpiar filtros">✕</a>
        <?php endif; ?>
      </form>

      <!-- Grilla de Tarjetas de Productos -->
      <div class="products-grid">
        <?php
        $hasProducts = false;
        while ($product = $products_stmt->fetch(PDO::FETCH_ASSOC)):
          $hasProducts = true;
        ?>
          <div class="product-card">
            <div class="product-image">
              <?php if(!empty($product['image'])): ?>
                <img src="uploads/<?= htmlspecialchars($product['image']) ?>" alt="<?=
htmlspecialchars($product['name']) ?>">
              <?php else: ?>
                <div class="no-image">📷 Sin Foto</div>
              <?php endif; ?>
              <span class="product-badge"><?= htmlspecialchars($product['category_name'])
?></span>
            </div>

            <div class="product-details">
              <h3><?= htmlspecialchars($product['name']) ?></h3>
              <p class="product-desc"><?= htmlspecialchars($product['description']) ?></p>

              <div class="product-footer">
```

```
            <div>
              <span class="product-price">$<?= number_format($product['price'], 2)
?></span>
              <span class="product-stock <?= $product['stock'] < 5 ? 'low-stock' : '' ?>">
                Stock: <?= $product['stock'] ?>
              </span>
            </div>

            <a href="index.php?action=delete&id=<?= $product['id'] ?>"
               class="btn-delete"
               onclick="return confirm('¿Seguro que deseas eliminar este producto?')"
               title="Eliminar producto">
              🗑️
            </a>
          </div>
        </div>
      </div>
      <?php endwhile; ?>

      <?php if (!$hasProducts): ?>
        <div class="empty-state">
          <span class="empty-icon">🔍</span>
          <p>No se encontraron productos registrados.</p>
        </div>
      <?php endif; ?>
    </div>

  </div>
 </main>

</body>
</html>
```

6. Estilos CSS (`styles.css`)

CSS

```
:root {
  --font-family: 'Plus Jakarta Sans', sans-serif;
  --bg-main: #0b0f19;
  --bg-card: rgba(22, 31, 48, 0.7);
  --bg-card-hover: rgba(30, 41, 64, 0.85);
  --border-color: rgba(255, 255, 255, 0.08);
  --primary: #6366f1;
  --primary-hover: #4f46e5;
  --text-main: #f8fafc;
```

```
  --text-muted: #94a3b8;
  --success: #10b981;
  --danger: #ef4444;
  --radius-sm: 8px;
  --radius-md: 16px;
  --shadow-main: 0 20px 40px -15px rgba(0, 0, 0, 0.5);
  --transition: all 0.3s ease;
}

* { margin: 0; padding: 0; box-sizing: border-box; }

body {
  font-family: var(--font-family);
  background-color: var(--bg-main);
  color: var(--text-main);
  min-height: 100vh;
  line-height: 1.5;
  background-image:
    radial-gradient(at 0% 0%, rgba(99, 102, 241, 0.12) 0px, transparent 50%),
    radial-gradient(at 100% 100%, rgba(168, 85, 247, 0.12) 0px, transparent 50%);
}

.container { max-width: 1200px; margin: 0 auto; padding: 0 24px 60px; }

/* Navbar y Hero */
.navbar { display: flex; justify-content: space-between; align-items: center; padding: 16px
40px; background: rgba(11, 15, 25, 0.8); backdrop-filter: blur(12px); border-bottom: 1px solid
var(--border-color); position: sticky; top: 0; z-index: 100; }
.logo { font-size: 1.2rem; font-weight: 700; display: flex; align-items: center; gap: 8px; }
.badge-mini { font-size: 0.7rem; background: var(--primary); padding: 2px 6px; border-radius:
4px; }
.nav-info { font-size: 0.85rem; color: var(--text-muted); display: flex; align-items: center; gap:
8px; }
.status-dot { width: 8px; height: 8px; background: var(--success); border-radius: 50%;
display: inline-block; }

.hero { text-align: center; padding: 36px 20px 24px; max-width: 700px; margin: 0 auto; }
.badge { display: inline-block; padding: 6px 16px; border-radius: 50px; background: rgba(99,
102, 241, 0.1); border: 1px solid rgba(99, 102, 241, 0.3); color: #818cf8; font-size: 0.85rem;
font-weight: 600; margin-bottom: 12px; }
.hero h1 { font-size: 2.2rem; font-weight: 700; margin-bottom: 8px; }
.gradient-text { background: linear-gradient(135deg, #818cf8 0%, #c084fc 100%);
-webkit-background-clip: text; -webkit-text-fill-color: transparent; }
.hero-subtitle { color: var(--text-muted); font-size: 0.95rem; }

.alert { padding: 12px 20px; border-radius: var(--radius-sm); margin-bottom: 24px; text-align:
center; font-weight: 500; }
```

```
.alert-success { background: rgba(16, 185, 129, 0.15); border: 1px solid var(--success); color: #6ee7b7; }
.alert-error { background: rgba(239, 68, 68, 0.15); border: 1px solid var(--danger); color: #fca5a5; }

/* Grid principal */
.app-grid { display: grid; grid-template-columns: 360px 1fr; gap: 24px; align-items: start; }
.card { background: var(--bg-card); backdrop-filter: blur(16px); border: 1px solid var(--border-color); border-radius: var(--radius-md); padding: 24px; box-shadow: var(--shadow-main); }
.card-header { margin-bottom: 20px; }
.card-header h2 { font-size: 1.15rem; font-weight: 600; }
.card-header p { font-size: 0.85rem; color: var(--text-muted); }

/* Formulario */
.contact-form { display: flex; flex-direction: column; gap: 14px; }
.form-group { display: flex; flex-direction: column; gap: 6px; }
.form-row { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; }
.form-group label { font-size: 0.85rem; font-weight: 600; color: var(--text-muted); }

input, select, textarea {
  font-family: inherit; background: rgba(11, 15, 25, 0.6); border: 1px solid var(--border-color); color: var(--text-main); padding: 10px 14px; border-radius: var(--radius-sm); font-size: 0.9rem; outline: none; transition: var(--transition);
}
input:focus, select:focus, textarea:focus { border-color: var(--primary); box-shadow: 0 0 0 3px rgba(99, 102, 241, 0.2); }
select option { background-color: var(--bg-main); }

/* Botones */
.btn { font-family: inherit; font-weight: 600; border-radius: var(--radius-sm); padding: 10px 16px; border: none; cursor: pointer; transition: var(--transition); display: inline-flex; align-items: center; justify-content: center; text-decoration: none; }
.btn-primary { background: var(--primary); color: white; }
.btn-primary:hover { background: var(--primary-hover); }
.btn-secondary { background: rgba(255, 255, 255, 0.08); color: var(--text-main); border: 1px solid var(--border-color); }
.btn-secondary:hover { background: rgba(255, 255, 255, 0.15); }
.btn-block { width: 100%; }
.btn-icon { background: transparent; color: var(--text-muted); border: 1px solid var(--border-color); padding: 10px 14px; }

/* Filtros y Productos */
.filter-bar { display: flex; gap: 8px; margin-bottom: 20px; }
.filter-bar input { flex: 1; }

.products-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(220px, 1fr)); gap: 16px; }
```

```
.product-card { background: rgba(11, 15, 25, 0.5); border: 1px solid var(--border-color); border-radius: var(--radius-sm); overflow: hidden; display: flex; flex-direction: column; transition: var(--transition); }
.product-card:hover { border-color: rgba(99, 102, 241, 0.4); transform: translateY(-3px); }

.product-image { height: 140px; background: rgba(255, 255, 255, 0.03); position: relative; display: flex; align-items: center; justify-content: center; }
.product-image img { width: 100%; height: 100%; object-fit: cover; }
.no-image { color: var(--text-muted); font-size: 0.85rem; }

.product-badge { position: absolute; top: 8px; right: 8px; background: rgba(11, 15, 25, 0.8); backdrop-filter: blur(4px); font-size: 0.7rem; padding: 3px 8px; border-radius: 12px; font-weight: 600; border: 1px solid var(--border-color); }

.product-details { padding: 14px; flex: 1; display: flex; flex-direction: column; justify-content: space-between; }
.product-details h3 { font-size: 0.95rem; font-weight: 600; margin-bottom: 4px; }
.product-desc { font-size: 0.8rem; color: var(--text-muted); margin-bottom: 12px; display: -webkit-box; -webkit-line-clamp: 2; -webkit-box-orient: vertical; overflow: hidden; }

.product-footer { display: flex; justify-content: space-between; align-items: flex-end; }
.product-price { display: block; font-size: 1.1rem; font-weight: 700; color: #818cf8; }
.product-stock { display: block; font-size: 0.75rem; color: var(--text-muted); }
.low-stock { color: #fca5a5; font-weight: 600; }

.btn-delete { text-decoration: none; font-size: 0.9rem; padding: 4px 8px; border-radius: 4px; transition: var(--transition); opacity: 0.7; }
.btn-delete:hover { opacity: 1; background: rgba(239, 68, 68, 0.2); }

.empty-state { grid-column: 1 / -1; text-align: center; padding: 40px 0; color: var(--text-muted); }
.empty-icon { font-size: 2rem; display: block; margin-bottom: 8px; }

@media (max-width: 900px) {
  .app-grid { grid-template-columns: 1fr; }
}
```

🎯 Conceptos Clave para tus Estudiantes

1. **`enctype="multipart/form-data"`**: Atributo fundamental en el formulario HTML para permitir el envío de archivos adjuntos (imágenes).
2. **Variable global `$_FILES`**: Recibe los metadatos del archivo subido (`name`, `tmp_name`, `error`, `size`).
3. **`move_uploaded_file()`**: Función que traslada el archivo de la carpeta temporal del servidor a la carpeta final `uploads/`.

4. **Relación de tablas (Foreign Key)**: `LEFT JOIN categories c ON p.category_id = c.id` relaciona la tabla `products` con `categories` para obtener el nombre de la categoría en la vista.
5. **Filtros acumulativos**: La consulta SQL construye condicionales dinámicas (`WHERE 1=1 AND ...`) según los parámetros pasados en el buscador.